

Национальный исследовательский университет ИТМО
Факультет прикладных компьютерных наук
Факультет прикладных компьютерных наук

Новокшанов Евгений Андреевич

Поддержание базы автономных систем на основе данных протокола BGP

Магистерская диссертация

Допущена к защите.
Зав. кафедрой:
— —

Научный руководитель:
научный руководитель Белявцев И. П.

Рецензент:
рецензент —

Санкт-Петербург
2026

ITMO University
Faculty of Applied Computer Science
Faculty of Applied Computer Science

Evgeny Novokshanov

Maintaining the Autonomous Systems Database Based on BGP Protocol Data

Graduation Thesis

Admitted for defence.

Head of the chair:

--

Scientific supervisor:
supervisor Ivan Belyavtsev

Reviewer:
reviewer —

Saint Petersburg
2026

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	7
СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ	9
ВВЕДЕНИЕ	10
Актуальность темы исследования	10
Степень разработанности темы	11
Решаемая проблема	11
Объект и предмет исследования	12
Цель и задачи работы	12
Методы исследования	13
Методологическая и теоретическая основа	13
Научная новизна	13
Положения, выносимые на защиту	14
Степень достоверности и апробация результатов	15
Практическая значимость	15
Структура работы	15
1. Предметная область междоменной маршрутизации и BGP	17
1.1. Автономные системы и междоменная маршрутизация	17
1.2. BGP как базовый протокол междоменной маршрутизации	17
1.3. Multiprotocol extensions и поддержка нескольких семейств адресов	18
1.4. CIDR и longest prefix match как фундамент задачи	19
1.5. Route Refresh и повторная синхронизация состояния	19
1.6. BGP Monitoring Protocol и наблюдаемость состояния	20
2. Динамика BGP и постановка проблемы актуальности состояния	21
2.1. Нестабильность и пересылка избыточных обновлений	21
2.2. Теоретический взгляд: задача устойчивых путей	21
2.3. Роль публичных коллекторов маршрутов	22
2.4. Где пакетная модель упирается в потолок	22

2.5.	Локальная динамика и flap damping	23
2.6.	Системные следствия для прикладного сервиса	24
3.	Выбор структуры хранения для in-memory таблицы префиксов	25
3.1.	Семейства структур для longest prefix match	25
3.2.	Adaptive Radix Tree как mutable in-memory индекс	25
3.3.	Почему ART выбран в этой работе	26
4.	Постановка задачи исследования	28
4.1.	Исследовательская гипотеза	28
4.2.	Функциональные требования	29
4.3.	Нефункциональные требования	29
4.4.	Критерии успеха	30
5.	Архитектура разработанного микросервиса	31
5.1.	Общая архитектурная идея	31
5.2.	Модель взаимодействия с GoBGP	32
5.3.	Пользовательский gRPC API	33
5.4.	Два режима хранения: legacy и новый	33
5.5.	Эксплуатационные ограничения	33
5.6.	Вынесение логики в отдельный микросервис как основной архитектурный результат	35
6.	Алгоритмы начального снимка и потоковой актуализации	36
6.1.	Начальный снимок как bootstrap	36
6.2.	Разрешение перекрывающихся префиксов	36
6.3.	Замена полного состояния	37
6.4.	Потоковая обработка изменений	37
6.5.	Пересинхронизация после ошибок	37
6.6.	Модель состояния и корректность	38
7.	Реализация шардированного ART-хранилища	40
7.1.	Общая идея шардирования	40
7.2.	Внутреннее представление ключа	40

7.3. Lookup по IP-адресу	41
7.4. Upsert и delete	41
7.5. Dump полного состояния	42
7.6. Контракт <code>prefixStore</code>	42
7.7. Целевая операция и интерпретация результатов	42
7.8. Согласованность параллельного доступа	43
7.9. Поведение во время ресинхронизации	44
8. Методика и организация экспериментов	45
8.1. Цель экспериментальной части	45
8.2. Сравнимые варианты	45
8.3. Конфигурация стенда	46
8.4. Наборы данных и сценарии	46
8.5. Почему именно эти метрики достаточны	47
8.6. Ограничения экспериментальной методики	47
9. Результаты экспериментов и их интерпретация	49
9.1. Полная выгрузка	49
9.2. Инкрементальное обновление	49
9.3. Lookup по IP	50
9.4. Влияние числа шардов	51
9.5. Поведение IPv6	51
9.6. Сводный итог сравнения	52
9.7. Совокупная стоимость потокового профиля нагрузки	53
10. Практические ограничения и направления развития	55
10.1. Ограничения текущей реализации	55
10.2. Направления развития	55
ЗАКЛЮЧЕНИЕ	57
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	59
СПИСОК ИЛЛЮСТРАТИВНОГО МАТЕРИАЛА	63

ПРИЛОЖЕНИЕ А. Развёрнутое сравнение альтернативных структур LPM	66
ПРИЛОЖЕНИЕ Б. Расширенная методика экспериментов и угрозы валидности	74
ПРИЛОЖЕНИЕ В. Надёжность, API-семантика и эксплуатационные сценарии	79
ПРИЛОЖЕНИЕ Г. Практическая интеграция и инфраструктурный контекст	85

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей работе используются следующие термины с соответствующими определениями.

Автономная система (AS) – совокупность сетевых префиксов, управляемых одним административным субъектом и анонсируемых наружу по единой маршрутной политике; идентифицируется числом ASN (Autonomous System Number).

BGP (Border Gateway Protocol) – протокол обмена маршрутной информацией между автономными системами, описанный в RFC 4271 [20].

BMP (BGP Monitoring Protocol) – протокол наблюдения за состоянием BGP-сеансов и содержимым таблиц маршрутизации [23].

CIDR (Classless Inter-Domain Routing) – модель бесклассовой адресации, в которой принадлежность адреса префиксу описывается парой «адрес/длина маски» [8].

Длиннейшее совпадение префикса (Longest Prefix Match, LPM) – задача поиска по IP-адресу префикса наибольшей длины, которому этот адрес принадлежит.

Полный снимок маршрутов (snapshot) – единомоментный набор действующих BGP-маршрутов, формирующих полное начальное состояние таблицы.

Поток обновлений (stream) – последовательность отдельных BGP-сообщений типа update/withdraw, изменяющих состояние таблицы после загрузки начального снимка.

Adaptive Radix Tree (ART) – адаптивное префиксное дерево с переменным размером узлов, описанное в работе Лейса и др. [14].

GoBGP – открытая реализация BGP-демона на языке Go, предоставляющая внешнее gRPC-API для управления и наблюдения.

gRPC – система удалённого вызова процедур поверх HTTP/2, используемая для взаимодействия микросервиса с GoBGP и с клиентами.

BIFIT MITIGATOR – продукт компании, обеспечивающий защиту от распределённых атак отказа в обслуживании.

BIFIT COLLECTOR – продукт для сбора и анализа сетевого трафика крупного оператора связи.

SLA (Service Level Agreement) – соглашение об уровне обслуживания, фиксирующее количественные требования к качеству работы сервиса.

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

API – application programming interface

ART – Adaptive Radix Tree

AS – автономная система

ASN – Autonomous System Number

BGP – Border Gateway Protocol

BMP – BGP Monitoring Protocol

CIDR – Classless Inter-Domain Routing

CPU – центральный процессор (central processing unit)

DDoS – distributed denial of service

gRPC – gRPC Remote Procedure Calls

IP – Internet Protocol

IPv4 / IPv6 – четвёртая и шестая версии IP

LPM – longest prefix match

MP-BGP – multiprotocol BGP

MRT – multi-threaded routing toolkit (формат дампов)

NLRI – network layer reachability information

RIB – routing information base

RIS – Routing Information Service (RIPE NCC)

SLA – service level agreement

ВКР – выпускная квалификационная работа

ВВЕДЕНИЕ

Актуальность темы исследования

Современная сетевая аналитика, фильтрация распределённых атак и телеметрия операторских сетей всё чаще опираются на маршрутный контекст адресов. Главный элемент этого контекста – принадлежность IP-адреса автономной системе: по ASN проще строить политики фильтрации, размечать поток и сопоставлять события с операторской топологией.

Результаты работы внедряются в инфраструктуру продуктов BIFIT MITIGATOR (защита от DDoS) и BIFIT COLLECTOR (анализ операторского трафика). Оба продукта должны опираться на одну и ту же таблицу *IP-префикс* $\rightarrow$ *ASN*, выведенную из полной таблицы маршрутизации BGP; внешние геолокационные справочники для режима реального времени не подходят. До начала работы логика построения и обновления этой таблицы была встроена в BIFIT COLLECTOR, а внутреннее хранилище *ranger* обновлялось по таймеру полной заменой снимка: потоковое применение одиночных изменений к in-memory базе не поддерживалось, а второй продукт (MITIGATOR) не мог переиспользовать ту же реализацию без дублирования кода и данных.

Естественным первичным источником актуальных данных остаётся сам протокол BGP [20]. Чтобы использовать его в прикладном контуре, нужно получить начальный снимок RIB, непрерывно обрабатывать update-сообщения, поддерживать IPv4 и IPv6 и подобрать in-memory структуру, в которой дешёво одновременно точечный поиск (LPM) и частые локальные правки – иначе поток обновлений останется декларацией.

Для задач, где допустимы часы задержки между изменением маршрутов и обновлением локальной копии, достаточно внешних справочников. У высоконагруженных сервисов иной запас: минуты расхождения с глобальной таблицей оборачиваются ложными срабатываниями и пропусками трафика [13, 6]. Отсюда требование к секундной свежести и к архитектуре, где обновление не сводится к периодической полной перестройке таблицы.

Степень разработанности темы

В мировой литературе вопросы динамики BGP и стабильности маршрутов изучаются с конца 1990-х: сюда относятся работы Лабовица о маршрутной нестабильности и времени сходимости BGP [13, 6], формальная постановка задачи устойчивых путей у Гриффина и др. [9], а также эмпирические исследования глобальной таблицы по данным RIPE RIS и RouteViews [19, 21, 1]. Алгоритмическая сторона longest prefix match имеет столь же длинную историю: PATRICIA-деревья [15], LC-trie Нильссона и Карлссона [17], controlled prefix expansion Шринивасана [24], Tree Bitmap Итертона [7] и Poptrie Асаи [3]. Adaptive Radix Tree предложен Лейсом и др. в 2013 году [14], но рассматривается там в контексте СУБД, а не как прикладное хранилище маршрутных данных. Архитектурная парадигма «snapshot + stream» проработана в литературе по системам обработки потоков [12, 5, 2, 11]. Задача переноса этой парадигмы на BGP-производное состояние при одновременном выделении функции сопоставления «префикс $\rightarrow$ ASN» в отдельный сервис с общим контрактом для нескольких продуктов, насколько известно автору, в открытых публикациях системно не ставилась.

Решаемая проблема

Прикладному контуру нужна единая точка истины для пары *префикс $\rightarrow$ ASN*, доступная нескольким продуктам и способная удерживать лаг порядка секунд. Прежняя схема сочетала встроенность логики в один продукт, опрос по таймеру и полную замену таблицы в *ranger*: при таком устройстве ни общий сервис для MITIGATOR и COLLECTOR, ни потоковое in-memory обновление без перестройки всей таблицы получить нельзя. Проблема, решаемая в работе, – вынести сопоставление в автономный микросервис с общим gRPC API и обеспечить штатный режим «начальный снимок + поток правок» к in-memory состоянию без отказа от прикладно приемлемого времени чтения.

Объект и предмет исследования

Объектом исследования являются программные средства поддержки актуального отображения сетевых префиксов в автономные системы по данным BGP.

Предметом исследования являются архитектурные решения по выделению функции сопоставления «префикс $\rightarrow$ ASN» в отдельный микросервис и методы потоковой актуализации его in-memory таблицы по данным BGP, включая выбор изменяемого (mutable) внутреннего хранилища, поддерживающего LPM и инкрементальные правки.

Цель и задачи работы

Цель работы – разработать и исследовать микросервис, который поддерживает актуальную таблицу соответствия *IP-префикс $\rightarrow$ ASN* по данным BGP в режиме реального времени.

Для достижения цели в работе решаются такие задачи:

1. изучить предметную область междоменной маршрутизации, свойства BGP и проблему свежести маршрутного состояния;
2. проанализировать известные подходы к LPM и структурам хранения префиксных таблиц;
3. сформулировать функциональные и нефункциональные требования к сервису, работающему по схеме «начальный снимок + поток обновлений»;
4. спроектировать архитектуру микросервиса и интерфейсы его взаимодействия с источником маршрутных данных и потребителями;
5. реализовать загрузку начального снимка и потоковую обработку обновлений с восстановлением после разрыва соединения;
6. реализовать шардированное in-memory хранилище на базе Adaptive Radix Tree с операциями поиска, вставки, удаления и полной выгрузки;

7. провести сравнительный эксперимент с прежним вариантом и оценить компромиссы между временем чтения, скоростью обновления и стоимостью полной выгрузки.

Методы исследования

В работе применяются методы системного анализа предметной области, сравнительного анализа алгоритмов и структур данных, эксперимент в форме воспроизводимых микробенчмарков на одном и том же стенде, а также аналитическое моделирование совокупной CPU-нагрузки для двух режимов работы. Каждый эксперимент по несколько раз воспроизводится, а полученные показатели сводятся в таблицы с одинаковым набором столбцов для двух конкурирующих реализаций.

Методологическая и теоретическая основа исследования

Методологическую базу составляют действующие RFC по BGP и его расширениям [20, 16, 8, 4, 18, 23], а также фундаментальные работы по динамике BGP [13, 6, 9]. Теоретической основой алгоритмической части служат работы по префиксным деревьям и LPM [15, 17, 22, 10, 24, 7, 3], а также оригинальная статья по Adaptive Radix Tree [14]. Архитектурный шаблон «начальный снимок + поток обновлений» взят из практики систем обработки потоков и распределённых данных [12, 5, 2, 11].

Научная новизна

Научная новизна работы состоит не в новом сетевом протоколе, а в постановке и решении прикладной задачи инфраструктурного уровня для продуктовой линейки BIFIT. Во-первых, функция сопоставления *IP-префикс* → *ASN* вынесена из состава BIFIT COLLECTOR в автономный микросервис с единым gRPC контрактом для нескольких потребителей – это меняет границы ответственности и устраняет необходимость дублировать логику между MITIGATOR и COLLECTOR. Во-вторых, реализован переход от периодической полной замены таблицы к потоковому применению BGP-событий

к in-memory базе по шаблону «начальный снимок + поток обновлений». В-третьих, для практической осуществимости потока выбрано и внедрено шардированное in-memory ядро на базе Adaptive Radix Tree [14]: структура обеспечивает дешёвые точечные правки, LPM-запросы, полную выгрузку и рассылку подписчикам, хотя в маршрутной литературе ART чаще обсуждается в контексте СУБД, а не BGP-состояния.

Положения, выносимые на защиту

1. Функция сопоставления «префикс $\rightarrow$ ASN» выделена в отдельный микросервис с общим gRPC API для нескольких продуктов; граница сервиса совпадает с границей предметной задачи, а не с границей одного из потребителей.
2. Архитектурный шаблон «начальный снимок + поток обновлений», адаптированный к источнику BGP-данных, обеспечивает потоковое поддержание in-memory таблицы без полной перестройки при каждом изменении маршрута.
3. Шардированное in-memory хранилище на основе Adaptive Radix Tree для пары «префикс $\rightarrow$ ASN» поддерживает LPM, точечные правки и полную выгрузку и служит технической основой для пункта 2.
4. Алгоритм восстановления состояния после разрыва с источником, сводящий пересинхронизацию к повторной загрузке снимка с последующим переходом в режим потока.
5. Эмпирическая оценка преимущества потокового варианта по операции инкрементального обновления и аналитическая модель совокупной CPU-стоимости двух backend-реализаций под профили нагрузки; численные пределы, при которых пакетная схема перестаёт быть эксплуатационно осуществимой.

Степень достоверности и апробация результатов

Достоверность результатов обеспечивается тем, что все эксперименты проведены на одном и том же стенде с фиксированной конфигурацией CPU, оперативной памяти и версии Go-runtime, а измерения выполнены стандартным механизмом `go test -bench` на нескольких размерах таблицы и нескольких профилях нагрузки. Конфигурация стенда и набор данных детально описаны в §8 и приложении Б. Апробация результатов проведена в инфраструктуре компании БИФИТ: сервис включён в сборки продуктов BIFIT MITIGATOR и BIFIT COLLECTOR и используется как единый источник пары «префикс → ASN» для нескольких внутренних потребителей.

Практическая значимость

Практическая значимость определяется применимостью результата как инфраструктурного сервиса в системах сетевой аналитики, защиты трафика и операторской отчётности, где важны:

- секундная свежесть таблицы маршрутов;
- единая точка обслуживания нескольких клиентов через общий API;
- согласованное и воспроизводимое состояние при перезапусках;
- малая стоимость частых инкрементальных правок;
- унификация источника данных и снижение стоимости поддержки нескольких продуктов компании.

Структура работы

Работа содержит введение, десять глав основной части, заключение, список использованных источников из 25 наименований и четыре приложения. Главы 1–2 посвящены теоретическим основам BGP и динамике междоменной маршрутизации; глава 3 – выбору in-memory структуры хранения префиксной таблицы с опорой на критерии LPM (глава не является самоцелью,

а готовит обоснование backend-решения). В главе 4 формулируется задача исследования. Главы 5–7 описывают архитектуру микросервиса, алгоритмы потоковой актуализации и реализацию шардированного ART-хранилища. Главы 8–9 посвящены методике и результатам сравнительного эксперимента. В главе 10 разобраны ограничения текущей реализации и направления развития. В заключении основные выводы. Приложения содержат расширенный сравнительный анализ структур LPM, развёрнутую методику экспериментов с обсуждением угроз валидности и описание API-семантики и эксплуатационных сценариев сервиса.

1. Предметная область междоменной маршрутизации и BGP

1.1. Автономные системы и междоменная маршрутизация

Интернет представляет собой объединение большого числа независимых сетей, принадлежащих различным организациям: операторам связи, облачным платформам, корпорациям, университетам, государственным структурам и другим участникам. На уровне административного управления эти сети объединяются в автономные системы (Autonomous Systems, AS). Каждая автономная система управляется единым субъектом и реализует собственную политику маршрутизации.

Междоменная маршрутизация отличается от внутридоменной тем, что её цель не ограничивается поиском кратчайшего пути. Здесь ключевую роль играют политические и экономические ограничения: приоритет партнёрских связей, экспортные и импортные политики, ограничения по префиксам и соображения отказоустойчивости. Именно поэтому BGP, в отличие от типичных IGP-протоколов, является policy-aware протоколом и работает не как алгоритм минимизации расстояния, а как механизм обмена достижимостью с учётом локальных политик [20, 9].

1.2. BGP как базовый протокол междоменной маршрутизации

Согласно RFC 4271 [20], BGP-4 является междоменным протоколом маршрутизации, основная задача которого состоит в обмене информацией о достижимости сетевых префиксов между BGP-speaking системами. Маршрутная информация в BGP включает не только факт достижимости префикса, но и последовательность автономных систем (AS-path), через которые эта достижимость проходит. Это позволяет одновременно выявлять петли маршрутизации и принимать policy-based решения о выборе лучшего маршрута.

Для целей настоящей работы важны несколько свойств BGP.

Во-первых, BGP оперирует префиксами, а не отдельными адресами. Сле-

довательно, задача сопоставления IP-адреса автономной системе в конечном счёте сводится к longest prefix match по актуальному набору объявленных префиксов.

Во-вторых, BGP изначально поддерживает classless routing и CIDR [8]: маршруты различной длины маски могут сосуществовать, перекрываться и конкурировать за роль лучшего маршрута для конкретного адреса.

В-третьих, BGP представляет состояние маршрутизации не как статическую таблицу, а как непрерывно эволюционирующее множество объявлений (announcements) и withdraw-событий. Это означает, что программная система, желающая использовать BGP как источник истины, не может ограничиться редким чтением снимка без риска работы с устаревшим представлением.

1.3. Multiprotocol extensions и поддержка нескольких семейств адресов

RFC 4760 [16] вводит расширения BGP-4 для переноса информации о достижимости нескольких сетевых протоколов одновременно. Для этого используются пары $\langle AFI, SAFI \rangle$ (Address Family Identifier и Subsequent Address Family Identifier) и специальные атрибуты MP_REACH_NLRI и MP_UNREACH_NLRI. Для рассматриваемой задачи это означает, что прикладной сервис должен корректно работать как минимум с двумя основными случаями:

- IPv4 unicast ($AFI = 1, SAFI = 1$);
- IPv6 unicast ($AFI = 2, SAFI = 1$).

Различие между IPv4 и IPv6 важно не только на уровне протокола, но и на уровне внутреннего поиска. Для IPv4 максимальная длина префикса составляет 32 бита, для IPv6 – 128 бит. Соответственно, реализация longest prefix match, основанная на переборе возможных длин маски или на байтовой навигации по структуре данных, имеет принципиально различные характеристики на этих двух семействах адресов; разница проявляется как в стоимости отдельного lookup, так и в общем числе кандидатных длин маски, которые может потребоваться проверить.

1.4. CIDR и longest prefix match как фундамент задачи

RFC 4632 [8] закрепляет практику Classless Inter-Domain Routing и фиксирует переход от классовой адресации к префиксной как ключевой механизм ограничения роста глобального маршрутизационного состояния. В classless-модели значение имеет не «класс сети», а явная пара *префикс* и *длина маски*.

Для прикладного сервиса это означает следующее. Если для одного и того же адресного пространства одновременно объявлено несколько перекрывающихся префиксов, то принадлежность конкретного IP-адреса определяется по наиболее специфичному (longest) из них. Простого словаря «адрес → значение» здесь принципиально недостаточно. Внутреннее хранилище обязано либо нативно поддерживать LPM, либо предоставлять структуру, поверх которой LPM может быть эффективно реализован.

1.5. Route Refresh и повторная синхронизация состояния

RFC 2918 [4] описывает механизм Route Refresh Capability, позволяющий BGP-speaker запрашивать переобъявление соответствующего Adj-RIB-Out у соседа без полного перезапуска BGP-сессии. RFC 7313 [18] расширяет этот механизм, вводя маркеры начала и завершения refresh-последовательности (Beginning of RR, End of RR), что позволяет более корректно и менее разрушительно проверять и корректировать несогласованное состояние.

Для сервиса, работающего по схеме «снимок + поток обновлений», это особенно важно. Поточная модель почти неизбежно сталкивается с вопросом ресинхронизации:

- что делать после разрыва соединения с источником данных;
- как убедиться, что между двумя моментами наблюдения не были потеряны withdraw-события;
- как безопасно вернуть состояние к согласованному виду без полной остановки и перезапуска всей системы.

Механизмы Route Refresh позволяют рассматривать такие сценарии не как ad-hoc инженерный костыль, а как легитимную часть протокольной модели BGP.

Дополнительный родственный сюжет – BGP route flap damping [25]. Хотя данная работа не реализует damping явно, его теоретический смысл, состоящий в подавлении нестабильности и сглаживании церемоний объявления/отзыва, важен для понимания того, почему обработка update-стрима не может быть наивной.

1.6. BGP Monitoring Protocol и наблюдаемость состояния

RFC 7854 [23] определяет BGP Monitoring Protocol (BMP) как специальный протокол для мониторинга BGP-сессий. В контексте настоящей работы BMP интересен не как непосредственно используемый транспорт, а как иллюстрация общей архитектурной идеи: маршрутизационное состояние имеет смысл собирать, наблюдать и передавать как отдельный поток данных, а не только как побочный эффект работы маршрутизатора.

Хотя в реализации, рассматриваемой в работе, интеграция строится не через BMP, а через GoBGP gRPC API, концептуально оба подхода принадлежат одному классу решений: отделение маршрутизационного plane от сервисов, использующих его как источник данных.

2. Динамика BGP и постановка проблемы актуальности состояния

2.1. Нестабильность и пересылка избыточных обновлений

Серия работ Лабовица и соавторов на рубеже 2000-х показала, что междоменная маршрутизация в реальной сети ведёт себя совсем не как статическая структура. В *Internet Routing Instability* [13] зафиксировано, что объём BGP-обновлений в среднем кратно превосходит интуитивный сценарий «редкие, осмысленные изменения». Существенная доля сообщений повторная, избыточная или возникает из-за патологий смежных маршрутизаторов. Любой прикладной сервис, привязанный к BGP-состоянию, обязан рассчитывать на непрерывный churn как на штатный режим, а не как на аномалию.

В *Delayed Internet Routing Convergence* [6] та же картина дополнена временным срезом: после изменения топологии глобальная сходимость занимает минуты, а в худших сценариях – значительно больше. Для прикладной системы, опирающейся на отображение *префикс* $\rightarrow$ *ASN*, неопределённость возникает на двух уровнях:

1. собственно динамика BGP, заданная распределённой природой протокола и не поддающаяся одностороннему контролю;
2. дополнительная задержка локального обновления таблицы внутри самой прикладной системы.

Первое нельзя устранить – его остаётся только принять как свойство среды. Второе целиком определяется архитектурой сервиса, и именно его снижением занимается настоящая работа.

2.2. Теоретический взгляд: задача устойчивых путей

Гриффин, Шеферд и Вилфонг [9] формализовали работу BGP как Stable Paths Problem (SPP). Модель показывает, что протокол ищет локально устойчивый выбор маршрутов в присутствии политик, и этот процесс в

общем случае не обязан сходиться к единому глобальному оптимуму. Для прикладной части отсюда следуют два наблюдения.

Во-первых, у BGP нет «идеальной глобальной истины»: один и тот же префикс одновременно может выглядеть по-разному с точки зрения разных наблюдателей в зависимости от их положения в графе и от их собственных политик.

Во-вторых, если даже сам протокол достигает согласия только в пределе, то нет смысла строить прикладной сервис как редкий синхронный пересчёт. Гораздо ближе к реальности здесь модель, в которой состояние эволюционирует во времени и обновляется отдельными событиями.

2.3. Роль публичных коллекторов маршрутов

Платформы RIPE Routing Information Service [19] и RouteViews [21] многие годы служат основными источниками данных как для академических исследований BGP, так и для эксплуатационной аналитики. RIS опирается на распределённую сеть коллекторов, собирающих маршрутные данные у добровольных партнёров на крупных точках обмена трафиком и через multihop-сеансы. RouteViews предоставляет доступ к текущим и историческим данным коллекторов и к архивам в формате MRT.

Для настоящей работы важно, что именно эти платформы давно живут в двойной модели данных: периодические дампы таблиц плюс непрерывный поток обновлений. Иначе говоря, даже на исследовательском уровне схема «снимок + поток» считается естественным способом организации маршрутных данных. Хорошая иллюстрация – механизм RIS Beacons [1], в котором заранее известные префиксы объявляются и отзываются по расписанию, превращая BGP в контролируемый эксперимент.

2.4. Где пакетная модель упирается в потолок

Допустим, прикладная система раз в несколько часов целиком перестраивает таблицу *префикс* $\rightarrow$ *ASN* по BGP full view. У такого подхода есть очевидные плюсы: не нужно отслеживать инкрементальные события, не нужно

следить за порядком обновлений, а внутреннее хранилище можно оптимизировать под чтение и под редкую целикомую замену.

В динамической среде BGP такая схема упирается сразу в несколько ограничений:

1. между фактическим изменением маршрута и его появлением в локальной таблице возникает лаг; для систем реального времени лаг в часы — это уже работа с устаревшей картиной мира;
2. чем чаще приходится перестраивать таблицу, тем дороже это обходится: стоимость одной перестройки определяется размером таблицы n , а не объёмом фактических изменений;
3. локальная правка одного префикса требует работы, сопоставимой по объёму с полной сборкой таблицы заново;
4. с ростом числа маршрутов (сегодня IPv4 уже превысил миллион, а IPv6 — сотни тысяч префиксов) и частоты их изменений пакетная схема начинает не укладываться в требования по задержке и по потреблению CPU.

Отсюда естественный переход к другой схеме: начальный снимок нужен только при старте, а дальше актуальность таблицы поддерживается потоком обновлений.

2.5. Локальная динамика и flap damping

Динамика BGP не сводится к «полезным» изменениям топологии. Заметная часть update-трафика порождается локальными нестабильностями — короткоживущими событиями, повторными withdraw/announce одного и того же префикса, последствиями переключения на резервные пути. RFC 2439 [25] описывает классический механизм route flap damping, который подавляет излишне нестабильные маршруты экспоненциально убывающим штрафом.

Прикладному сервису flap damping не нужно реализовывать самостоятельно, но он остаётся важной иллюстрацией двух более общих наблюдений:

- поток BGP-обновлений несёт значительную долю избыточной информации;
- потоковая часть сервиса должна выдерживать всплески, многократно превышающие средний поток событий.

Это закладывает мягкое, но конкретное требование к структуре хранения: она обязана переживать частые локальные правки без катастрофического роста стоимости и без полной перестройки. Это требование вместе с семантикой lookup определит выбор алгоритмов в главах 4 и 7.

2.6. Системные следствия для прикладного сервиса

Соберу свойства, разобранные выше, в четыре положения о проектировании.

1. Модель «редкий пересчёт всего состояния» концептуально конфликтует с природой BGP: протокол не статичен, а непрерывно эволюционирует.
2. Рассинхронизация неизбежна; правильное проектирование закладывает дисциплину пересинхронизации как штатный режим, а не как обработку аварии.
3. Стоимость одного события на стороне сервиса должна оставаться небольшой даже при высокой интенсивности потока, иначе потоковая модель теряет смысл.
4. Семантика Lookup не должна зависеть от того, в какой момент произошло конкретное обновление: видимое клиенту состояние обязано соответствовать какой-то определённой точке последовательности применённых событий.

Эти четыре положения становятся техническими требованиями к сервису, формализованными в главе 4 как функциональные и нефункциональные ограничения.

3. Выбор структуры хранения для in-memory таблицы префиксов

Глава не ставит целью разработку нового алгоритма longest prefix match: LPM – необходимый критерий при выборе структуры для таблицы *префикс* $\rightarrow$ *ASN*, согласованной с потоком BGP-событий. Ниже кратко сопоставляются известные семейства; развёрнутое сравнение и таблицы критериев приведены в приложении А.

3.1. Семейства структур для longest prefix match

Исторически LPM связывают с trie-подобными представлениями: PATRICIA [15] убирает длинные цепочки узлов с одним потомком; LC-trie [17] сжимает плотные уровни ради низкой задержки lookup. Другой класс – хеширование, бинарный поиск по длинам масок и controlled prefix expansion [22, 10, 24] – ориентирован на минимум обращений к памяти и аппаратную реализуемость. Tree Bitmap [7] и Poptrie [3] задают ориентиры для программных и гибридных реализаций быстрого forwarding lookup.

Для микросервиса на CPU иного профиля: доминирует не пакетная обработка на ASIC, а *изменяемое* in-мемогу хранилище с частыми точечными правками, полной выгрузкой, шардированием и простой интеграцией в Go-runtime. Структуры «только для lookup» или «только для bulk replace» плохо стыкуются с требованием потокового обновления; поэтому выбор делается в пространстве компромиссов между чтением, инкрементом и эксплуатационной простотой, а не по одной микрометрике LPM.

3.2. Adaptive Radix Tree как mutable in-memory индекс

Работа Leis, Kemper и Neumann [14] изначально относится к области in-memory indexing в базах данных, а не к маршрутизации. Тем не менее ART представляет особый интерес и для рассматриваемой здесь задачи. В ART внутренние узлы имеют адаптивный формат: Node4, Node16, Node48 и Node256. Это означает, что структура подстраивается под локальную плот-

ность ветвления и стремится одновременно быть и компактной, и быстрой.

Ключевые свойства ART, важные для нашей задачи:

- поддержка поиска по байтовому представлению ключа;
- сравнительно эффективные вставки и удаления;
- естественное хранение данных в отсортированном порядке;
- адаптивное потребление памяти, согласованное с реальным числом потомков узлов.

Важно отметить, что ART не является классической «канонической» структурой для *longest prefix match* в маршрутизаторной литературе. Его сила здесь проявляется в другом: он хорошо подходит как *mutable ordered index* общего назначения, который можно использовать как основу для *in-memory* хранилища префиксов в сервисном контуре.

3.3. Почему ART выбран в этой работе

Выбор ART в данной работе основан не на утверждении, что ART является оптимальным алгоритмом LPM для всех возможных систем, а на следующих практических соображениях.

1. Требуется поддержка инкрементальных обновлений, поскольку именно они становятся основным режимом работы при переходе к *streaming*-модели.
2. Требуется предсказуемая и достаточно компактная *in-memory* структура, легко интегрируемая в Go-runtime сервиса.
3. Требуется возможность построения полного дампа без полного пересчёта таблицы и без введения отдельной параллельной структуры.
4. Требуется сравнительно простая реализация и интеграция в микросервис, в том числе при отладке и сопровождении.

5. Требуется возможность независимой работы нескольких шардов под разные участки ключевого пространства.

На этом основании ART рассматривается как практический компромисс между lookup-ориентированными префиксными структурами и mutable ordered index. Расширенное сравнение альтернатив и обоснование выбора по критериям приведено в приложении А.

4. Постановка задачи исследования

Инфраструктурная боль, из которой выросла работа, двоякая. С одной стороны, логика построения таблицы *IP-префикс* $\rightarrow$ *ASN* жила внутри BIFIT COLLECTOR, тогда как BIFIT MITIGATOR нуждался в тех же данных: без отдельного сервиса неизбежны дублирование и расхождение копий. С другой стороны, backend *ranger* обновлялся по таймеру полной заменой снимка; одиночная вставка сводилась к перестройке всей структуры, так что поток BGP-событий нельзя было применять к in-memory базе по одному событию без катастрофической цены.

Сервис, рассматриваемый в работе, обслуживает несколько внутренних потребителей и должен отвечать на три типа запросов:

- к какой автономной системе относится конкретный IP-адрес;
- каков полный текущий снимок таблицы соответствия префиксов автономным системам;
- какие изменения в таблице произошли с момента подписки клиента.

Прежняя связка «встроенная логика + *ranger*» удовлетворяла части требований к чтению, но не позволяла ни разделить продукты по границе сервиса, ни сделать потоковое обновление базовым режимом при лаге порядка секунд.

4.1. Исследовательская гипотеза

Опираясь на свойства BGP (глава 2) и критерии выбора in-memory структуры (глава 3), сформулирую гипотезу так:

если вынести функцию сопоставления IP-префикс $\rightarrow$ ASN из состава одного продукта в автономный микросервис с общим gRPC API для нескольких потребителей, перейти от периодической полной замены таблицы к потоковому применению BGP-событий к in-memory базе по схеме «начальный снимок + поток обновлений» и реализовать внутреннее хранилище на основе

шардированного Adaptive Radix Tree, то таблицу можно поддерживать актуальной в реальном времени и снизить стоимость одного инкрементального обновления на несколько порядков по сравнению с пакетной моделью на ranger.

Архитектурный шаблон «снимок + поток» – частный случай более общей stateful-streaming парадигмы, описанной у Акидау и соавторов [5]; он хорошо сочетается с лог-ориентированными платформами Kafka [12] и Flink [2] и согласуется с трактовкой состояния как функции последовательности событий поверх базового снимка [11].

4.2. Функциональные требования

1. Сервис должен уметь подключаться к источнику маршрутного состояния (узлу GoBGP) и считывать полный снимок IPv4 и IPv6 маршрутов.
2. Сервис должен уметь принимать поток update-событий и отражать его во внутреннем состоянии.
3. Сервис должен предоставлять API для точечного lookup по IP-адресу.
4. Сервис должен предоставлять API для получения полного снимка таблицы.
5. Сервис должен предоставлять API подписки на инкрементальные изменения.
6. Сервис должен корректно обрабатывать вставки, удаления и повторное построение состояния после ресинхронизации.

4.3. Нефункциональные требования

1. Одиночное обновление не должно требовать полной перестройки всей таблицы.
2. Чтение из таблицы должно оставаться достаточно быстрым для прикладного использования.

3. Полная выгрузка может быть медленнее, чем в batch-оптимизированном решении, но должна оставаться практически приемлемой.
4. Архитектура должна позволять одновременно обслуживать нескольких клиентов через единый API.
5. Реализация должна быть достаточно прозрачной и сопровождаемой, включая возможность профилирования и наблюдения внутреннего состояния.

4.4. Критерии успеха

Критерии, по которым результат работы должен признаваться успешным, вытекают из исследовательской гипотезы и требований:

1. стоимость отражения одного изменения маршрута уменьшена не менее чем на несколько порядков;
2. время точечного lookup сохранено в прикладно-приемлемых пределах;
3. возможность построения полного дампа сохранена и остаётся работоспособной для целевого размера таблиц;
4. потоковая модель работает как основной операционный сценарий, а не как факультативное дополнение к batch-режиму.

5. Архитектура разработанного микросервиса

5.1. Общая архитектурная идея

Центральный архитектурный результат – не столько выбор конкретного дерева, сколько выделение сопоставления «префикс $\rightarrow$ ASN» в отдельный сетевой процесс между источником BGP и продуктами компании. Сервис занимает промежуточное положение между узлом GoBGP и потребителями; последние подключаются по одному и тому же gRPC контракту, независимо от того, реализован backend на *ranger* или на шардированном ART. В качестве источника используется GoBGP с gRPC API; в качестве потребителей – BIFIT MITIGATOR и BIFIT COLLECTOR. Логически архитектура изображена на рис. 1.

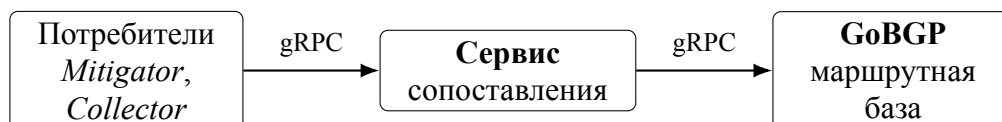


Рисунок 1 – Общая архитектура: сервис между потребителями и узлом GoBGP

На логическом уровне внутри сервиса можно выделить следующие компоненты:

- модуль конфигурации и запуска;
- gRPC-сервер пользовательского API;
- клиент GoBGP;
- доменный сервис синхронизации состояния;
- хранилище префиксов *prefixStore*;
- подсистема рассылки обновлений подписчикам (*subscriber hub*);
- вспомогательные механизмы диагностики и профилирования (*pprof*).

Такое разделение важно, потому что оно изолирует два разных типа ответственности: получение и актуализацию маршрутизационного состояния

с одной стороны и обслуживание внешних запросов с другой. Внутренние логические слои показаны на рис. 2.

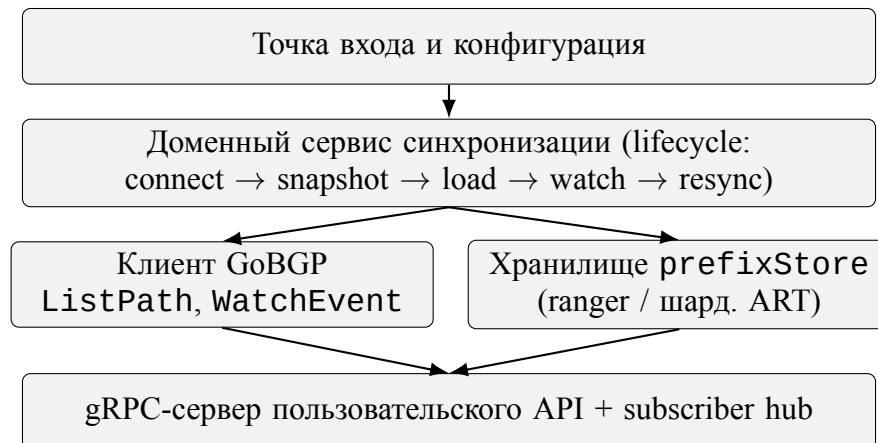


Рисунок 2 – Внутренние логические слои сервиса

5.2. Модель взаимодействия с GoBGP

С GoBGP сервис говорит по двум каналам.

Первый канал, `ListPath`, отдаёт полный снимок таблицы. Это операция начальной загрузки, но при необходимости она вызывается повторно – например, при пересинхронизации.

Второй канал, `WatchEvent`, поставляет поток событий: новые анонсы и отзывы маршрутов в том порядке, в котором их видит соседний BGP-роутер.

Получается жизненный цикл такого вида:

1. подключиться к GoBGP;
2. прочитать полный снимок;
3. нормализовать и загрузить его во внутреннее хранилище;
4. перейти в режим обработки потока обновлений;
5. при ошибке или разрыве – переподключиться и пересинхронизироваться.

По смыслу это та же схема «начальная загрузка от снимка плюс непрерывный поток», что и в распределённых системах обработки событий [5, 2].

5.3. Пользовательский gRPC API

Для внешних потребителей реализуются три основных операции.

`GetASNInfo` выполняет lookup по IP-адресу и возвращает информацию о найденном ASN и наиболее специфичном префиксе.

`GetPrefixesASTable` возвращает полный снимок таблицы соответствия префиксов автономным системам.

`SubscribePrefixASUpdates` позволяет клиенту получать непрерывный поток изменений после момента подписки.

Такой набор операций покрывает как *synchronous query*-сценарии, так и *reactive*-сценарии непрерывного наблюдения за изменениями. Подробное обсуждение семантики этих методов и поведения при отказах вынесено в приложение В.

5.4. Два режима хранения: legacy и новый

В процессе работы сервиса поддерживаются два варианта backend-хранилища (см. рис. 3):

- **ranger** – прежнее решение, оптимизированное под полную замену таблицы;
- **шардированный ART** – новое решение, поддерживающее инкрементальные модификации.

Наличие двух вариантов полезно не только для постепенной миграции, но и для экспериментов: оно позволяет на одном и том же сервисном контуре измерять компромиссы между *batch*-оптимизированным и *stream*-ориентированным подходами в идентичных условиях.

5.5. Эксплуатационные ограничения

При проектировании микросервиса с самого начала фиксируются несколько эксплуатационных ограничений, которые дальше определяют устройство внутренних механизмов.

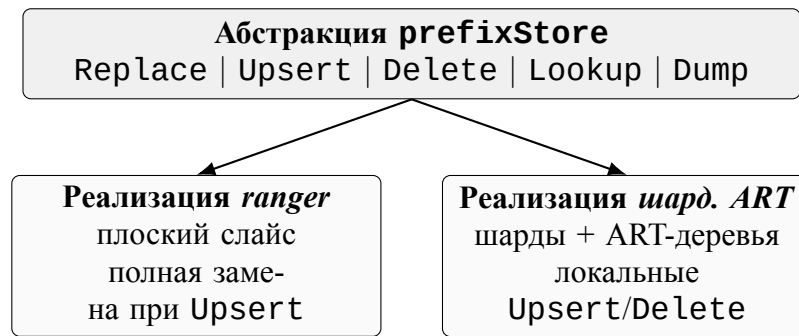


Рисунок 3 – Контракт `prefixStore` и две реализации

- **Один источник маршрутных данных.** Сервис подключается к одному узлу GoBGP. Это сознательное упрощение: оно делает модель состояния однозначной и снимает вопрос о конфликтном слиянии данных от нескольких BGP-роутеров. Архитектура расширению не препятствует, но текущая версия работает по правилу «один источник – один поток событий – одно внутреннее состояние».
- **Одна копия состояния на всех клиентах.** Сервис держит одну логическую копию маршрутной таблицы. Альтернативная схема с копией на каждого клиента потребовала бы либо лишнего расхода памяти, либо более сложной сериализации обновлений; ни то ни другое для рассматриваемых сценариев не оправдано.
- **Гарантии видимости.** После того как обновление применилось к хранилищу, оно становится видимым последующим читателям и подписчикам. Строгой глобальной упорядоченности видимости для разных читателей не требуется – это согласуется с моделью BGP, где распределённое состояние сходится во времени.
- **Время отклика.** `Lookup` остаётся синхронным вызовом в пределах одного gRPC-запроса. Тяжёлые операции (`GetPrefixesASTable`, подписка на поток) живут в отдельных RPC и не блокируют точечные запросы.

5.6. Вынесение логики в отдельный микросервис как основной архитектурный результат

До внедрения отдельного сервиса таблица «префикс → ASN» строилась и обновлялась внутри BIFIT COLLECTOR: второй продукт не мог опереться на ту же реализацию без копирования кода и состояния, а смена backend потребовала бы правок в прикладной логике коллектора. После выделения функции в микросервис MITIGATOR и COLLECTOR становятся клиентами одного контракта: граница процесса совпадает с границей предметной задачи – сопоставление адресов и префиксов ASN по данным BGP.

Такое разделение даёт и эксплуатационный эффект. Обновление алгоритма загрузки снимка, политики нормализации префиксов или типа хранилища (*ranger* против шардированного ART) локализуется в одном месте; потребители продолжают вызывать те же RPC. Согласованность данных между продуктами перестаёт зависеть от дублирования кода; сопровождение не распадается на две независимые реализации одной и той же таблицы.

6. Алгоритмы начального снимка и потоковой актуализации

6.1. Начальный снимок как bootstrap

При запуске сервис запрашивает полный снимок таблицы маршрутизации отдельно для IPv4 и IPv6. Такой разделённый подход упрощает как логическую обработку, так и последующее распараллеливание.

После получения маршрутов из GoBGP данные ещё не являются готовой таблицей $prefix \rightarrow ASN$. Причина в том, что исходное множество маршрутов может содержать перекрывающиеся префиксы, а прикладному сервису требуется построить непротиворечивое представление, в котором на каждую адресную область определён наблюдаемый ASN.

6.2. Разрешение перекрывающихся префиксов

Одной из важных задач при построении полной выгрузки является устранение неоднозначностей, связанных с перекрывающимися сетевыми префиксами. Если просто механически скопировать все маршруты в хранилище, то внешний пользователь при построении полной таблицы получит набор перекрывающихся диапазонов, который неудобен для многих downstream-сценариев.

Поэтому между этапом чтения маршрутов и этапом загрузки в store выполняется flattening-преобразование: исходный набор маршрутов приводится к непересекающемуся множеству диапазонов, в котором каждой области сопоставляется доминирующий ASN. На уровне идеи это близко к sweep-line обработке интервалов: события «начало диапазона» и «конец диапазона» обрабатываются в порядке адресов, а в каждой точке поддерживается множество активных префиксов с выбором наиболее специфичного из них.

Этот этап особенно важен для полной выгрузки и для repeatable snapshot semantics: клиенты получают не произвольный внутренний набор префиксов, а нормализованный срез состояния.

6.3. Замена полного состояния

После построения снимка сервис выполняет операцию `Replace` для выбранного backend-хранилища. Для `legacy`-режима это естественная операция, поскольку вся модель хранения основана на полной замене. Для `ART`-режима `Replace` тоже допустим, но используется только на этапе `bootstrap` и ресинхронизации, а не как нормальный способ обработки каждого `update`-события.

6.4. Поточковая обработка изменений

После начальной загрузки сервис подписывается на поток BGP-событий. Каждое событие интерпретируется как одна из двух базовых операций над `prefixStore`:

- `Upsert` маршрута;
- `Delete` маршрута.

В `stream-oriented` модели именно эти операции становятся главным рабочим режимом. Их обработка должна:

1. быть локальной, а не глобальной;
2. изменять только затронутый участок состояния;
3. сохранять корректность `lookup`;
4. порождать уведомления для подписчиков через `subscriber hub`.

6.5. Пересинхронизация после ошибок

Потоковая модель имеет естественный недостаток: если соединение оборвалось, нельзя безусловно предположить, что локальное состояние осталось согласованным с источником. Поэтому система использует стратегию *reconnect-and-resync*, изображённую как конечный автомат на рис. 4:

1. поток завершается или переходит в ошибку;

2. клиент закрывает соединение;
3. заново считывается полный снимок;
4. выполняется `Replace` для всего хранилища;
5. подписка на стрим открывается повторно.

С точки зрения чистоты модели это соответствует философии Route Refresh [4, 18] и bootstrap from snapshot [5]: поток трактуется как эволюция состояния относительно некоторой базовой точки, и если базовая точка стала недостоверной, она строится заново.

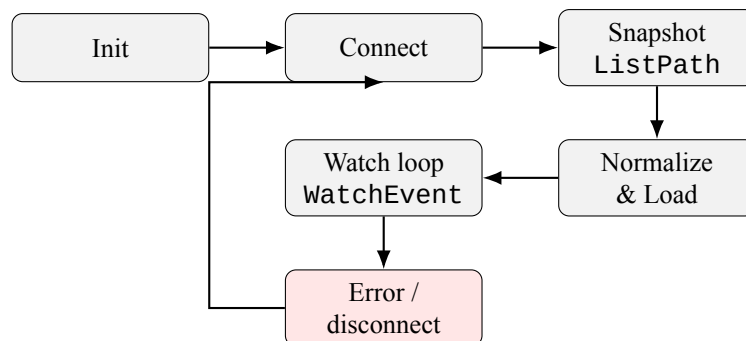


Рисунок 4 – Конечный автомат жизненного цикла *reconnect-and-resync*

6.6. Модель состояния и корректность

Снимок и поток удобно рассматривать не как противоположности, а как две фазы жизни одного и того же материализованного состояния. Снимок нужен при старте, при пересинхронизации и при выгрузке полной таблицы наружу. Поток поддерживает свежесть, переносит локальные правки и кормит подписчиков. Подробное эксплуатационное обсуждение этой двойственности вынесено в приложение В.

Порядок применения событий задаётся самой сессией с GoBGP: сервис обрабатывает их в том порядке, в котором они приходят. По терминологии работ по обработке потоков [5] это *processing-time*, а не *event-time*. Для прикладной задачи такой выбор разумен по двум причинам. Во-первых, истинное время события у самого исходного BGP-роутера сервису недоступно –

между ним и сервисом стоит GoBGP, агрегирующий поток. Во-вторых, важна не глобальная синхронизация по астрономическим часам, а локальная согласованность шагов: после применения события e_i и до применения e_{i+1} ответы на **Lookup** обязаны соответствовать состоянию после e_i . Эта гарантия обеспечивается дисциплиной работы с **prefixStore**: события обрабатываются по одному, а видимость их результата управляется блокировками внутри шардов.

Кратко состояние таблицы записывается как

$$S_n = f(S_0, e_1, e_2, \dots, e_n),$$

где S_0 — результат начальной загрузки, а $e_1, \dots, e_n$ — последовательно применённые BGP-события. Любой ответ **Lookup** клиента C в момент t_C соответствует ровно одному из S_k , $k \leq n(t_C)$. Полная выгрузка **GetPrefixesASTable** эквивалентна выбору одного S_k . Подписка **SubscribePrefixASUpdates** даёт поток дельт, который позволяет любому клиенту, начав с S_k , восстановить $S_{k+1}, S_{k+2}, \dots$. Из этой записи видно главное: снимок и поток связаны единой семантикой, а **Replace** после пересинхронизации — не аномалия, а переустановка референсной точки S_0 при сохранении того же типа функции f .

7. Реализация шардированного ART-хранилища

7.1. Общая идея шардирования

Одно из ограничений любой общей in-memory структуры заключается в конкуренции за блокировки. Если все операции чтения и записи проходят через одно дерево и один lock, то потоковая модель быстро упрётся в contention. Поэтому в работе применяется шардирование.

Ключ префикса хешируется (используется FNV-32a), и значение хеша по модулю числа шардов определяет, в какой шард он попадёт. Каждый шард содержит собственное ART-дерево и собственный `sync.RWMutex`. Такой подход даёт два преимущества:

- обновления разных префиксов, попадающих в разные шарды, не конкурируют напрямую;
- чтение и запись локализуются внутри одного шарда.

Устройство шардированного хранилища изображено на рис. 5.

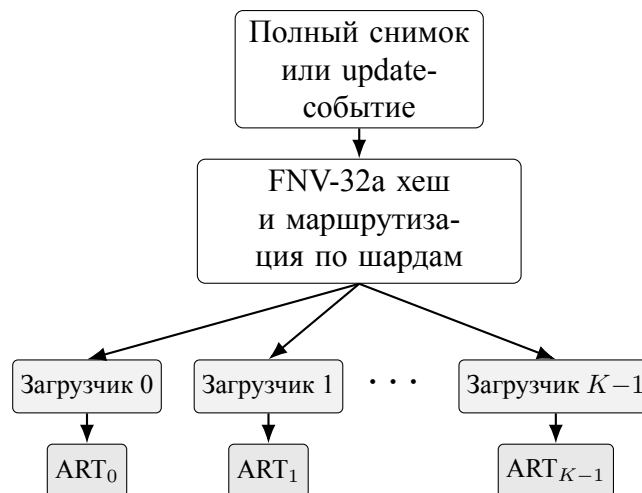


Рисунок 5 – Устройство шардированного ART-хранилища

7.2. Внутреннее представление ключа

Для хранения в ART IP-префикс переводится в байтовое представление, удобное для лексикографической навигации по дереву. Этот этап важен, потому что ART работает на последовательности байтов, а не на абстрактном

объекте «префикс». Соответственно, корректное кодирование ключа определяет корректность дальнейших операций поиска и обхода [14]. В реализации используется фиксированная длина кодирования с явной длиной маски, что упрощает дальнейший longest prefix match.

7.3. Lookup по IP-адресу

Точечный lookup для внешнего клиента реализует longest prefix match по текущему состоянию хранилища. В зависимости от способа организации ключей и навигации по дереву возможны разные стратегии:

- хранение всех префиксов и поиск наиболее специфичного кандидата;
- последовательная проверка длин масок (до 33 для IPv4 и до 129 для IPv6);
- более сложная навигация по radix-структуре с удержанием последнего совпадения вдоль пути.

В текущей реализации lookup имеет приемлемые характеристики для IPv4 и более высокую стоимость для IPv6, что связано с особенностями перебора масок и большей глубиной адресного пространства. Возможные направления оптимизации обсуждаются в главе 10.

7.4. Upsert и delete

Главное достоинство нового backend-хранилища проявляется именно на операциях обновления. **Upsert** затрагивает один шард и локально модифицирует его дерево; **Delete** обрабатывает withdraw-событие точно так же – локально, без перестройки остальных шардов. Это принципиально отличается от поведения batch-структуры, в которой каждое изменение маршрута приводило к полной перестройке таблицы.

7.5. Dump полного состояния

Полная выгрузка в ART-режиме реализуется как обход всех шардов и сборка результатов. Это естественно делает Dump дороже, чем в структуре, которая держит уже готовый линейный массив записей. По сути, ART-подход платит за mutability дополнительной ценой обхода, аллокаций и (опционально) последующей сортировки. В реализации сортировка управляется параметром `ASMAP_STORE_ART_DUMP_SORT`, что позволяет в сценариях, где порядок не важен, экономить заметную долю времени на больших таблицах.

Эта особенность не является дефектом реализации как таковым. Она отражает фундаментальный компромисс: структура, оптимизированная под частые инкрементальные обновления, редко оказывается одновременно лучшей и для операции «отдать всё содержимое сразу».

7.6. Контракт `prefixStore`

С точки зрения исследовательской работы важна не сама конкретная реализация, а наличие явного контракта `prefixStore`, общего для двух backend-вариантов. Контракт задан минимальным набором операций, описанных в табл. 1; это делает сравнение реализаций честным и сводит экспериментальный анализ к сопоставлению поведения двух конкретных воплощений одного и того же интерфейса.

7.7. Целевая операция и интерпретация результатов

Оценивать новое хранилище по одному только времени Dump или по latency точечного Lookup некорректно. Целевая функция работы другая – сделать потоковую актуализацию реализуемой на практике. Главной метрикой здесь становится стоимость `Upsert` и `Delete`, а `Dump` и `Lookup` играют роль «расходов на остальные сценарии». Эту установку важно удерживать при чтении главы 9, где приведены эмпирические результаты.

Таблица 1 – Контракт `prefixStore` и принципиальное поведение реализаций

Операция	Реализация <i>ranger</i>	Реализация <i>шард. ART</i>
<code>Replace</code>	полная замена слайса	параллельная загрузка по шардам
<code>Upsert</code>	пересборка всего слайса и полная замена ($O(n)$)	локальная модификация одного шарда ($O(\log n/K)$ амортизированно)
<code>Delete</code>	пересборка всего слайса	локальное удаление в одном шарде
<code>Lookup</code>	longest prefix match по линейной структуре	longest prefix match по ART-дереву соответствующего шарда
<code>Dump</code>	копирование готового слайса (1 аллокация)	обход всех K шардов, опционально сортировка

7.8. Согласованность параллельного доступа

Шардирование само по себе не определяет модель согласованности; оно лишь снижает contention между независимыми операциями. Чтобы `Lookup` и `Upsert` безопасно сосуществовали внутри одного шарда, в реализации используется стандартный read-write-mutex. Все операции записи (`Upsert`, `Delete`, локальная фаза `Replace`) удерживают write-lock соответствующего шарда; `Lookup` удерживает read-lock того шарда, в котором происходит поиск.

Такая дисциплина даёт следующие гарантии:

- внутри одного шарда параллельные `Lookup` выполняются без взаимного блокирования;
- модификация одного шарда не блокирует операции в других шардах;
- `Dump` не требует глобального write-lock и реализуется последовательным захватом read-lock каждого шарда либо параллельным обходом независимых шардов с локальными read-lock.

Важно отметить, что согласованность `Dump` в этой модели является *шардовой*, а не глобальной snapshot-согласованностью во всём дереве: каждое

значение, появившееся в дампе, корректно относительно своего шарда в момент чтения. Для прикладного сценария получения полной таблицы это допустимо, поскольку клиент интерпретирует результат как «текущее представление сервиса», а не как атомарный одномоментный срез всех шардов.

При необходимости более сильной семантики можно либо ввести дополнительный механизм глобального snapshot (например, версионирование шардов), либо ограничиться тем, что глобальный «строгий» snapshot по построению не нужен в логике downstream-клиентов: они и так строят своё состояние как $S_0 + \Delta$ через подписку.

7.9. Поведение во время ресинхронизации

Отдельный важный режим работы – ресинхронизация. В этот момент сервис должен:

1. получить новый снимок;
2. подготовить из него корректное внутреннее представление;
3. заменить текущее состояние store на новое;
4. возобновить обработку потока обновлений.

В шардированной реализации эта последовательность выполняется как скоординированная замена содержимого всех шардов; параллельный Lookup, попадающий в момент перестроения, либо обслуживается из частично уже обновлённого состояния, либо ожидает завершения write-фазы конкретного шарда. Аномалии «исчезновения» ранее существовавшего префикса в этот момент возможны, но они являются ожидаемым следствием ресинхронизации и кратковременны. Учитывая, что ресинхронизация происходит редко (только при сбоях или после длительных пауз), такая модель является разумным инженерным компромиссом.

8. Методика и организация экспериментов

8.1. Цель экспериментальной части

Экспериментальная часть должна ответить на вопрос, подтверждается ли выдвинутая гипотеза — в части возможности потокового обновления `in-memory` базы после выделения микросервиса. Для этого нужно измерить, как новый и старый backend ведут себя на наборе операций, соответствующих реальным сценариям использования сервиса:

- полная выгрузка таблицы (`Dump`);
- одиночное обновление маршрута (`Upsert`);
- точечный поиск по IP-адресу (`Lookup`);
- влияние числа шардов на ключевые операции (*shard sweep*);
- различия между IPv4 и IPv6.

8.2. Сравнимые варианты

В опытах сравниваются:

- **ranger** — legacy backend на базе библиотеки ranger;
- **шард. ART** — новый backend на базе шардированного Adaptive Radix Tree.

Это важный методический момент: сравнение выполняется не между двумя абстрактными структурами данных из учебника, а между двумя реальными backend-вариантами одного и того же сервиса с одним и тем же `prefixStore`-контрактом. Все различия, фиксируемые в эксперименте, можно интерпретировать как различия между batch-ориентированной и stream-ориентированной реализациями этого контракта.

8.3. Конфигурация стенда

Параметры экспериментального стенда приведены в табл. 2. Все измерения проводились на одной и той же физической машине, без использования контейнеров и виртуализации, с зафиксированным набором фоновых процессов.

Таблица 2 – Конфигурация стенда экспериментов

Параметр	Значение
CPU	AMD Ryzen AI 9 HX 370 (24 потока)
ОЗУ	32 ГБ LPDDR5X
Среда	Linux x86_64
Среда исполнения	Go 1.24+
Инструмент	go test -bench с теговой подсистемой bench, benchheavy
ART-параметры	32 шарда (по умолчанию), withDumpSort(false) в сценариях с большими n
Размеры таблиц	$n \in \{1\,000; 10\,000; 100\,000\}$
Семейства адресов	IPv4 /24 (искусственные диапазоны 10.x.y.0/24, 11.x.y.0/24) и IPv6 /64 в 2001:db8::/32
Параметры запуска	count=2, benchtime=400ms / 500ms

8.4. Наборы данных и сценарии

Для измерений используются синтетические наборы префиксов, а также сценарии, имитирующие реалистичную нагрузку. В частности:

- таблицы различного размера, вплоть до 10^5 префиксов;
- отдельные наборы для IPv4 и IPv6;
- сценарии полного Dump;
- сценарии одиночного Upsert поверх уже загруженной таблицы;
- сценарии Lookup;
- sweep по числу шардов $K \in \{1, 8, 16, 32, 64\}$.

Такой дизайн эксперимента позволяет оценить не только «среднюю температуру», но и понять, какие операции становятся *bottleneck* для каждой реализации.

8.5. Почему именно эти метрики достаточны

Для данной работы важно подчеркнуть, что метрики выбираются не случайно, а отражают конкретные эксплуатационные сценарии.

Операция *Dump* соответствует сценарию получения полного снимка внешним потребителем и внутреннему *bootstrap/resync*.

Операция *Upsert* соответствует основной сущности потокового режима: локальному отражению одного BGP-изменения. Именно её стоимость определяет, осмыслен ли переход от полной пересборки таблицы к потоку; измерение *Upsert* – прямая проверка этой части гипотезы, а не «бенчмарк ради ART».

Операция *Lookup* задаёт дополнительный разрез: приемлемость задержки точечного запроса при выбранном хранилище (включая реализацию LPM).

Sweep по *shard count* показывает, где проходит граница между полезной параллельностью и издержками чрезмерного дробления состояния.

Именно эта совокупность метрик позволяет проверить, достигнута ли цель перехода к *stream-oriented* архитектуре, и подтвердить или опровергнуть выдвинутую гипотезу.

8.6. Ограничения экспериментальной методики

Эксперименты в работе ориентированы преимущественно на микро- и мезоуровневую производительность *in-memory* операций. Это означает, что они не охватывают в полной мере:

- сетевые задержки в внешних *gRPC*-вызовах;
- долгосрочную динамику при реальном потоке *update*-событий за большие интервалы времени;

- поведение под многоклиентской нагрузкой на пользовательский API;
- влияние GC и allocator pressure на длительном uptime;
- воспроизведение реалистичного распределения длин префиксов из production BGP-таблиц.

Однако для цели данной работы такой уровень детализации достаточен: он позволяет проверить сочетание «отдельный микросервис + потоковое in-memory обновление» и сравнить два варианта backend по критически важным операциям. Подробное обсуждение угроз валидности и причин, по которым они не разрушают главный вывод, вынесено в приложение Б.

9. Результаты экспериментов и их интерпретация

9.1. Полная выгрузка

Измерения показывают, что legacy backend, основанный на ranger, значительно быстрее выполняет полную выгрузку таблицы. Это объясняется тем, что он концептуально хранит уже подготовленный линейный набор записей и может отдавать его почти как копию готового массива. Шардированный ART, напротив, должен обойти все шарды, декодировать ключи, собрать результат и (по умолчанию) отсортировать его.

Конкретные численные результаты приведены в табл. 3. На малых таблицах ($n = 1000$, $n = 10000$) разница составляет два–три порядка, а на $n = 100000$ при выключенной сортировке разрыв сокращается, оставаясь, однако, значительным.

Таблица 3 – Бенчмарк Dump для IPv4 /24

Бэкенд (n)	Время	Память	Аллокаций
ranger, $n = 1000$	$\approx 2,6$ мкс/оп	8 KiB	1
ranger, $n = 10\,000$	≈ 30 мкс/оп	80 KiB	1
ranger, $n = 100\,000$	≈ 245 – 275 мкс/оп	784 KiB	1
ART, $n = 1000$	≈ 337 мкс/оп	≈ 270 KiB	≈ 3970
ART, $n = 10\,000$	$\approx 3,2$ мс/оп	$\approx 3,0$ MiB	$\approx 35\,000$
ART, $n = 100\,000^*$	$\approx 13,4$ – $13,8$ мс/оп	$\approx 33,4$ MiB	$\approx 351\,000$

* ART, $n = 100\,000$ – режим без сортировки дампа (`WithDumpSort(false)`).

На первый взгляд это может показаться аргументом против новой структуры. Однако такой вывод был бы неверным, потому что Dump не является главным рабочим режимом потоковой модели: цена 11–15 миллисекунд при 10^5 префиксов в абсолютных значениях остаётся эксплуатационно приемлемой.

9.2. Инкрементальное обновление

С точки зрения постановки задачи этот раздел – центральный: он показывает, что потоковое применение одиночных BGP-изменений к in-memory базе технически осуществимо, а не только объявлено в архитектуре. Именно

на операции одиночного **Upsert** новый подход демонстрирует решающее преимущество (табл. 4). У **ranger** каждый **Upsert** пересобирает весь список и вызывает **Replace**, что приводит к полной перестройке внутренней структуры. У **ART** та же операция выполняется локально в одном шарде, без вмешательства в остальные.

Таблица 4 – Бенчмарк одиночного **Upsert** поверх заполненной таблицы (IPv4)

n	ranger	ART (32 шарда)
1000	$\approx 3,0$ мс/оп	≈ 128 нс/оп
10 000	≈ 43 мс/оп	≈ 125 нс/оп
100 000	$\approx 587\text{--}616$ мс/оп	$\approx 165\text{--}169$ нс/оп

Разница достигает примерно **шести порядков** при $n = 10^5$: с ~ 600 мс/оп до ~ 170 нс/оп. Это главный эмпирический аргумент в пользу потоковой модели: без такой стоимости **Upsert** поддержание таблицы по одному событию оставалось бы декларацией. Новая архитектура качественно меняет допустимую модель эксплуатации: если при **ranger** поток BGP-изменений практически несовместим с локальным обновлением, то при шардированном **ART** инкремент становится естественным базовым режимом – уже внутри выделенного микросервиса с общим API.

9.3. Lookup по IP

Это вспомогательный разрез относительно центральной цели работы (микросервис и поток обновлений), но он фиксирует цену точечного чтения после выбора структуры. По lookup-операции картина оказывается более неоднородной. Для IPv4 старый backend показывает меньшую latency, а **ART** остаётся в зоне приемлемых, но более высоких значений. Для IPv6 разрыв становится ещё заметнее: текущая реализация **ART** выполняет lookup существенно медленнее (табл. 5). Объяснение в том, что **ART** при поиске перебирает все возможные длины масок – до 33 для IPv4 и до 129 для IPv6.

Это важный и честный результат, который не следует скрывать. Он показывает, что **ART backend** не является универсальным абсолютным победителем по всем метрикам. Однако он не опровергает гипотезу работы: цель

Таблица 5 – Бенчмарк Lookup (IPv4 /32 и IPv6 /128, $n \approx 10^5$)

Сценарий	ranger	ART (32 шарда)
IPv4 /32	151 нс/оп	400–500 нс/оп
IPv6 /128	168–178 нс/оп	3700–4000 нс/оп

состояла не в максимизации одной операции lookup любой ценой, а в достижении баланса между lookup и инкрементальной актуализацией. Возможные пути оптимизации lookup для ART рассматриваются в главе 10 и приложении Б.

9.4. Влияние числа шардов

Эксперименты показывают, что увеличение числа шардов не даёт линейного ускорения всех операций. Для Lookup выигрыш часто невелик или находится в пределах шума. Для Dump слишком большое число шардов может даже ухудшать результат из-за роста служебных издержек (табл. 6). Для update-сценариев умеренное шардирование оказывается полезным, потому что снижает конкуренцию между независимыми модификациями.

Таблица 6 – Sweep по числу шардов: Dump ART, IPv4, без сортировки

K (шардов)	$n = 5000$	$n = 100\,000$
1	–	$\approx 12,2\text{--}12,6$ мс/оп
8	$\approx 1,45\text{--}1,47$ мс/оп	$\approx 10,9\text{--}11,6$ мс/оп
16	$\approx 1,50\text{--}1,56$ мс/оп	$\approx 10,9\text{--}11,6$ мс/оп
32	$\approx 1,58\text{--}1,59$ мс/оп	$\approx 13,9\text{--}15,3$ мс/оп
64	$\approx 1,76\text{--}1,78$ мс/оп	$\approx 13,9\text{--}14,8$ мс/оп

Следовательно, число шардов является параметром настройки, а не правилом «чем больше, тем лучше». Практически разумной выглядит зона умеренного шардирования ($K \in [8; 32]$), а не экстремальные значения.

9.5. Поведение IPv6

IPv6 заслуживает отдельного внимания. По Dump разрыв между ranger и ART сохраняется на том же уровне, что и для IPv4: примерно 200–250 мкс

против 12,5–13,7 мс. По **Upsert** картина та же: ~ 889 мс у **ranger** против ~ 173 –182 нс у **ART**, что снова даёт разницу более чем в пять порядков. По **Lookup** **ranger** быстрее в десятки раз, что объясняется значительно большим числом перебираемых длин масок в текущей реализации **ART**. Этот результат важен как ограничение реализации, а не как свойство самого подхода.

9.6. Сводный итог сравнения

Сводка по сценариям приведена в табл. 7.

Таблица 7 – Итоговая сводка: какой backend выигрывает в каком сценарии

Сценарий	Победитель	Комментарий
Полная выгрузка	ranger	готовый линейный массив; ART платит за обход и аллокации
Одиночный Upsert	ART	5–6 порядков преимущества; решающий фактор для streaming-режима
Lookup IPv4	ranger	разница в 2–3 раза; для прикладного сервиса допустимо
Lookup IPv6	ranger	разрыв в десятки раз; ограничение текущей реализации LPM
Поток обновлений в целом	ART	batch-структура принципиально несовместима с моделью
Bootstrap / Replace	сопоставимо	обе реализации справляются за приемлемое время

Содержательно из сводки следуют четыре наблюдения:

1. прежнее решение лучше оптимизировано под пакетный сценарий «получить полный снимок и отдать целиком»;
2. новое решение – под потоковый сценарий «применить локальную правку»;
3. новый бэкенд платит за изменяемость более дорогими **Dump** и частью **Lookup**;
4. только новый бэкенд делает практически выполнимым непрерывное поддержание таблицы в актуальном состоянии.

Четвёртый пункт фиксирует главный системный вывод: потоковое обновление in-методу таблицы становится эксплуатационно реальной опцией. На этом фоне выделение функции в микросервис и единый gRPC контракт для нескольких продуктов – самостоятельный архитектурный слой; шардированный ART – средство, сделавшее этот поток дешёвым по CPU по сравнению с пакетной перестройкой ranger.

9.7. Совокупная стоимость потокового профиля нагрузки

Чтобы перевести микробенчмарки в эксплуатационно содержательную плоскость для уже выделенного микросервиса, полезно построить простую модель совокупной стоимости для типичных streaming-профилей. Рассмотрим установившийся режим, в котором:

- в системе хранится таблица из n префиксов ($n \in \{10^4; 10^5\}$);
- за единицу времени происходит λ инкрементальных обновлений (Upsert/Delete);
- клиенты выполняют μ запросов Lookup в секунду;
- операция полного Dump вызывается с частотой ν в секунду.

Тогда суммарную CPU-нагрузку на хранилище можно оценить как

$$C(\lambda, \mu, \nu) = \lambda \cdot t_{\text{upsert}}(n) + \mu \cdot t_{\text{lookup}}(n) + \nu \cdot t_{\text{dump}}(n),$$

где t_{upsert} , t_{lookup} и t_{dump} – это средние времена соответствующих операций, взятые из табл. 3–6.

Подставим типичные значения для $n = 10^5$:

- ranger: $t_{\text{upsert}} \approx 6,0 \cdot 10^{-1}$ с; $t_{\text{lookup}} \approx 1,5 \cdot 10^{-7}$ с; $t_{\text{dump}} \approx 2,6 \cdot 10^{-4}$ с;
- шард. ART: $t_{\text{upsert}} \approx 1,7 \cdot 10^{-7}$ с; $t_{\text{lookup}} \approx 4,5 \cdot 10^{-7}$ с; $t_{\text{dump}} \approx 1,4 \cdot 10^{-2}$ с.

Сравнение совокупной стоимости для двух представительных профилей нагрузки приведено в табл. 8.

Таблица 8 – Оценка совокупной CPU-стоимости в установившемся режиме при $n = 10^5$ (приблизённо, с/с = доля одного CPU-секунды на каждую секунду физического времени)

Сценарий	ranger	шард. ART
$\lambda=10, \mu=10^4, \nu=10^{-2}$	$\approx 6,00$ с/с	$\approx 0,0045$ с/с
$\lambda=10^3, \mu=10^5, \nu=10^{-2}$	≈ 600 с/с	$\approx 0,047$ с/с

Числа в табл. 8 имеют ясную интерпретацию. В обоих случаях ranger требует более чем одной полной CPU-секунды на каждую секунду физического времени уже при умеренной интенсивности обновлений; для второго профиля ему понадобились бы сотни ядер только для обработки изменений, что эксплуатационно невозможно. ART-вариант, напротив, в обоих случаях остаётся на уровне процентов от одного ядра, что и означает реальную возможность использовать его как online-обновляемый источник состояния.

Более того, в этой модели хорошо видно, что доминирующая статья расходов меняется. Для batch-варианта это всегда обновления (независимо от наличия читателей), для stream-варианта – скорее Lookup и редкие Dump. Это качественно меняет ту сторону системы, на которой имеет смысл проводить дальнейшую оптимизацию.

Если переложить те же цифры в плоскость SLA, картина проступает ещё резче. Пусть требуемая свежесть таблицы $T_{\text{stale}} \leq 1$ с. Чтобы удерживать такой лаг, пакетной модели пришлось бы перестраивать всю таблицу по несколько раз в секунду; при $n = 10^5$ это ~ 600 мс CPU-работы постоянно. Для потоковой модели то же требование выполняется само собой: каждое обновление применяется за единицы микросекунд. Эти два варианта находятся в качественно разных SLA-режимах – первый при умеренной интенсивности обновлений просто не способен удерживать секунднй лаг.

10. Практические ограничения и направления развития

10.1. Ограничения текущей реализации

Несмотря на положительные результаты по линии «микросервис + потоковое обновление», текущая реализация имеет ряд ограничений.

Во-первых, *lookup* для IPv6 в существующем варианте ART backend недостаточно оптимален. Это связано с выбранным способом *longest prefix match* (перебор по длинам масок) и представляет собой прямое направление для алгоритмической доработки поверх уже принятой архитектуры выделенного сервиса.

Во-вторых, *Dump* остаётся дороже, чем у *ranger*. В прикладном сценарии это допустимо, если полный снимок вызывается сравнительно редко, но при интенсивном использовании полного дампа может понадобиться дополнительный кэшированный *snapshot*-слой.

В-третьих, потоковая модель пока в основном рассматривается как последовательность *update*-событий из одного источника. Для более сложных *production*-сценариев могут понадобиться расширенные гарантии повторной доставки, дедупликации и диагностики пропущенных событий [12, 11].

10.2. Направления развития

Дальнейшие улучшения системы укладываются в три направления.

Lookup. Очевидный шаг – сделать LPM поверх ART более «родным», без полного перебора длин масок. Потенциально это включает хранение признака терминальности в узлах, удержание последнего валидного совпадения по ходу прохода и более компактное кодирование ключа, а также выделение *fast-path*-реализаций отдельно для IPv4 и IPv6. Работы *Tree Bitmap* [7] и *Poptrie* [3] показывают, насколько далеко можно продвинуться по этой оси.

Dump. Для редких, но дорогих полных выгрузок логично ввести кэшированный упорядоченный снимок, ленивую материализацию или потоковую выдачу содержимого вместо одного большого ответа. Если сервис начнёт ис-

пользоваться как поставщик периодических выгрузок, полезны и дифференциальные снимки для конкретных подписчиков.

Наблюдаемость. Расширение телеметрии и интеграция с ВМР-наблюдением [23] и публичными коллекторами [19, 21] позволят контролировать потери событий, сравнивать локальную таблицу с независимым внешним источником и оценивать свежесть состояния. Здесь же уместен механизм, аналогичный route flap damping [25], для подавления гиперактивных префиксов.

Шаблон «отдельный сервис с единым API + снимок + поток + шардированное in-memory ядро» переносим за пределы рассмотренной пары продуктов. Он применим в системах сетевой безопасности, аналитике трафика, операторской отчётности и сетевом инвентаре – везде, где нужно держать актуальный маршрутный контекст для принятия решений и не дублировать логику между потребителями.

ЗАКЛЮЧЕНИЕ

Работа посвящена поддержанию актуального соответствия *IP-префикс* $\rightarrow$ *ASN* по данным BGP для продуктов BIFIT MITIGATOR и BIFIT COLLECTOR. Главный результат – не отдельная микрооптимизация LPM, а сочетание архитектурного и алгоритмического уровней: общий микросервис и потоковое in-memory обновление таблицы.

Вынесение функции в микросервис. Логика построения и обновления таблицы перенесена из состава BIFIT COLLECTOR в автономный процесс с единым gRPC API для нескольких потребителей. Это устраняет дублирование между продуктами и фиксирует границу ответственности: сопоставление по данным BGP живёт в одном месте, независимо от выбранного backend (*ranger* или шардированный ART).

Потоковое обновление in-memory базы. Реализована операционная модель «начальный снимок + поток событий» от узла GoBGP: обновления применяются к одной общей копии состояния по одному событию, с автоматической пересинхронизацией после разрывов. На материале работ по динамике BGP [13, 6, 9] это согласуется с непрерывным характером маршрутного потока и несовместимостью с одной лишь периодической полной перестройкой.

Структура хранения и корректность снимка. После сжатого обзора критериев для префиксных структур (глава 3, приложение А) в качестве технической основы потока выбран и реализован шардированный Adaptive Radix Tree [14] с операциями *Replace*, *Upsert*, *Delete*, *Lookup* и *Dump*. Для внешних клиентов доступны *GetASNInfo*, *GetPrefixesASTable* и *SubscribePrefixASUpdates*. Полный снимок строится без перекрывающихся префиксов в выдаче.

Эксперимент и компромиссы. Сравнение на одном стенде двух реализаций одного *prefixStore*-контракта показало: при $n = 10^5$ шардированный ART уступает *ranger* по *Dump* (порядка 50 раз) и по *Lookup* для IPv4 в 2–3 раза, для IPv6 по *Lookup* – на порядок хуже из-за перебора длин масок; зато по *Upsert* преимущество составляет 5–6 порядков, то есть именно потоковое обновление становится эксплуатационно реальным, а пакетная схема при умеренной интенсивности обновлений требует кратных CPU-ядер только на

пересборку и не укладывается в секундный лаг.

Сопоставление с задачами работы: главы 1–2 дают теоретическую базу; глава 3 – обоснование выбора хранилища; глава 4 – требования и гипотеза; главы 5–7 – архитектура микросервиса, алгоритмы снимка и потока, реализация хранилища; главы 8–9 – методика и численные оценки.

Рекомендации: как единый источник пары «префикс → ASN» в режиме реального времени сервис с шардированным ART предпочтителен; если доминирует редкая полная выгрузка большой таблицы без потока, оправдан *ranger*; в смешанном профиле – гибрид с кэшем для *Dump*.

Перспективы: нативный LPM поверх ART без полного перебора масок (особенно IPv6), кэшированный упорядоченный снимок для *Dump*, интеграция с BMR [23] и коллекторами [19, 21, 1], проверки на production-подобных данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Aben Emile. RIS Beaconing - Shining a Light on BGP Dynamics. — 2023. — Access mode: <https://labs.ripe.net/author/emileaben/ris-beaconing-shining-a-light-on-bgp-dynamics/>.
- [2] Carbone Paris, Katsifodimos Asterios, Ewen Stephan, Markl Volker, Haridi Seif, and Tzoumas Kostas. Apache Flink: Stream and Batch Processing in a Single Engine // Bulletin of the IEEE Computer Society Technical Committee on Data Engineering. — 2015. — Vol. 36, no. 4. — P. 28–38. — Access mode: <https://asterios.katsifodimos.com/assets/publications/flink-deb.pdf>.
- [3] Asai Hirochika and Ohara Yasuhiro. Poptrie: A Compressed Trie with Population Count for Fast and Scalable Software IP Routing Table Lookup // Proceedings of ACM SIGCOMM. — 2015. — Access mode: <https://conferences.sigcomm.org/sigcomm/2015/pdf/papers/p57.pdf>.
- [4] Chen Enke. Route Refresh Capability for BGP-4. — RFC 2918. — 2000. — Access mode: <https://www.rfc-editor.org/rfc/rfc2918.html>.
- [5] Akidau Tyler, Bradshaw Robert, Chambers Craig, Chernyak Slava, Fernández-Moctezuma Rafael J., Lax Reuven, McVeety Sam, Mills Daniel, Perry Frances, Schmidt Eric, and Whittle Sam. The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing // Proceedings of the VLDB Endowment. — 2015. — Vol. 8, no. 12. — P. 1792–1803. — Access mode: <https://vldb.org/pvldb/vol8/p1792-Akidau.pdf>.
- [6] Labovitz Craig, Ahuja Abha, Bose Abhijit, and Jahanian Farnam. Delayed Internet Routing Convergence // Proceedings of ACM SIGCOMM. — 2000. — P. 175–187. — Access mode: <https://>

[//conferences.sigcomm.org/sigcomm/2000/conf/paper/sigcomm2000-5-2.pdf](https://conferences.sigcomm.org/sigcomm/2000/conf/paper/sigcomm2000-5-2.pdf).

- [7] Eatherton Will, Varghese George, and Dittia Zubin. Tree Bitmap: Hardware/Software IP Lookups with Incremental Updates // ACM SIGCOMM Computer Communication Review. — 2004. — Vol. 34, no. 2. — P. 97–122. — Access mode: <https://cseweb.ucsd.edu/~varghese/PAPERS/willpaper.pdf>.
- [8] Fuller Vince and Li Tony. Classless Inter-domain Routing (CIDR): The Internet Address Assignment and Aggregation Plan. — RFC 4632. — 2006. — Access mode: <https://www.rfc-editor.org/rfc/rfc4632.html>.
- [9] Griffin Timothy G., Shepherd F. Bruce, and Wilfong Gordon. The Stable Paths Problem and Interdomain Routing // IEEE/ACM Transactions on Networking. — 2002. — Vol. 10, no. 2. — P. 232–243. — Access mode: <https://www.cs.princeton.edu/~jrex/teaching/spring2005/reading/griffin02.pdf>.
- [10] Gupta Pankaj, Lin Steven, and McKeown Nick. Routing Lookups in Hardware at Memory Access Speeds // Proceedings of IEEE INFOCOM. — 1998. — Vol. 3. — P. 1240–1247.
- [11] Kleppmann Martin. Designing Data-Intensive Applications. — O'Reilly Media, 2017.
- [12] Kreps Jay, Narkhede Neha, and Rao Jun. Kafka: A Distributed Messaging System for Log Processing // Proceedings of NetDB. — 2011. — Access mode: <https://cs.uwaterloo.ca/~ssalihog/courses/papers/netdb11-final12.pdf>.
- [13] Labovitz Craig, Malan G. Robert, and Jahanian Farnam. Internet Routing Instability // Proceedings of ACM SIGCOMM. — 1997. — P. 115–126. — Access mode: <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/lmj97.pdf>.

- [14] Leis Viktor, Kemper Alfons, and Neumann Thomas. The Adaptive Radix Tree: ARTful Indexing for Main-Memory Databases // Proceedings of IEEE ICDE. — 2013. — P. 38–49. — Access mode: <https://db.in.tum.de/~leis/papers/ART.pdf>.
- [15] Morrison Donald R. PATRICIA - Practical Algorithm To Retrieve Information Coded in Alphanumeric // Journal of the ACM. — 1968. — Vol. 15, no. 4. — P. 514–534. — DOI 10.1145/321479.321481.
- [16] Bates Tony, Chandra Ravi, Katz Dave, and Rekhter Yakov. Multiprotocol Extensions for BGP-4. — RFC 4760. — 2007. — Access mode: <https://www.rfc-editor.org/rfc/rfc4760.html>.
- [17] Nilsson Stefan and Karlsson Gunnar. IP-address Lookup Using LC-tries // IEEE Journal on Selected Areas in Communications. — 1999. — Vol. 17, no. 6. — P. 1083–1092. — Открытый PDF — более ранняя публикация тех же авторов в Dr. Dobbs's Journal (авг. 1998); в списке приведена рецензируемая версия IEEE JSAC, DOI 10.1109/49.772439. Access mode: <https://www.gdt.id.au/~gdt/presentations/2020-09-03-gdt-routingtable/drddobbs199808--nilsson-karlsson--fast-ip-routing-with-l.pdf>.
- [18] Patel Keyur, Chen Enke, and Venkatachalapathy Balaji. Enhanced Route Refresh Capability for BGP-4. — RFC 7313. — 2014. — Access mode: <https://www.rfc-editor.org/rfc/rfc7313.html>.
- [19] RIPE NCC. Routing Information Service (RIS). — 2025. — Access mode: <https://www.ripe.net/analyse/internet-measurements/routing-information-service-ris>.
- [20] Rekhter Yakov, Li Tony, and Hares Susan. A Border Gateway Protocol 4 (BGP-4). — RFC 4271. — 2006. — Access mode: <https://www.rfc-editor.org/rfc/rfc4271.html>.

- [21] RouteViews Project. RouteViews API Documentation. — 2025. — Access mode: <https://api.routeviews.org/docs/>.
- [22] Waldvogel Marcel, Varghese George, Turner Jonathan S., and Plattner Bernhard. Scalable High Speed IP Routing Lookups // Proceedings of ACM SIGCOMM. — 1997. — P. 25–36. — Access mode: <https://kops.uni-konstanz.de/server/api/core/bitstreams/3aed3679-f4db-481c-b132-1775c0e0333a/content>.
- [23] Scudder John, Fernando Rex, and Stuart Stephen. BGP Monitoring Protocol (BMP). — RFC 7854. — 2016. — Access mode: <https://www.rfc-editor.org/rfc/rfc7854.html>.
- [24] Srinivasan Venkatachary and Varghese George. Fast Address Lookups Using Controlled Prefix Expansion // ACM Transactions on Computer Systems. — 1999. — Vol. 17, no. 1. — P. 1–40. — DOI 10.1145/296502.296503.
- [25] Villamizar Curtis, Chandra Ravi, and Govindan Ramesh. BGP Route Flap Damping. — RFC 2439. — 1998. — Access mode: <https://www.rfc-editor.org/rfc/rfc2439.html>.

СПИСОК ІЛЮСТРАТИВНОГО МАТЕРІАЛА

Рисунки

1.	Общая архитектура: сервис между потребителями и узлом GoBGP	31
2.	Внутренние логические слои сервиса	32
3.	Контракт <code>prefixStore</code> и две реализации	34
4.	Конечный автомат жизненного цикла <i>reconnect-and-resync</i> . .	38
5.	Устройство шардированного ART-хранилища	40

Таблицы

1.	Контракт <code>prefixStore</code> и принципиальное поведение реализаций	43
2.	Конфигурация стенда экспериментов	46
3.	Бенчмарк <code>Dump</code> для IPv4 /24	49
4.	Бенчмарк одиночного <code>Upsert</code> поверх заполненной таблицы (IPv4)	50
5.	Бенчмарк <code>Lookup</code> (IPv4 /32 и IPv6 /128, $n \approx 10^5$)	51
6.	Sweep по числу шардов: <code>Dump ART</code> , IPv4, без сортировки . . .	51
7.	Итоговая сводка: какой backend выигрывает в каком сценарии	52
8.	Оценка совокупной CPU-стоимости в установившемся режиме при $n = 10^5$ (приблизённо, с/с = доля одного CPU-секунды на каждую секунду физического времени)	54
9.	Качественное сравнение структур хранения по ключевым критериям	72

ПРИЛОЖЕНИЕ А. Развёрнутое сравнение альтернативных структур LPM

А.1. Зачем нужен отдельный сравнительный раздел

Одной из типичных слабых сторон прикладных магистерских работ является ситуация, когда автор сразу показывает выбранное решение и кратко отмечает, что «другие варианты менее удобны». Для инженерного отчёта этого иногда достаточно, но для диссертации требуется более строгая аргументация. Необходимо показать не только то, что выбранный вариант работает, но и то, что выбор был сделан в пространстве реальных альтернатив и по понятным критериям.

В настоящей работе сравнение альтернатив особенно важно, потому что выбор структуры хранения напрямую определяет архитектурную модель. Если выбрать структуру, идеально подходящую только для `lookup`, то можно получить великолепные микропоказатели чтения и провалиться на `update stream`. Если выбрать структуру, идеально подходящую только для `bulk replace`, то можно сохранить быстрый `Dump`, но так и не перейти к потоковой актуализации. Поэтому сравнительный анализ нужен не как формальность, а как основа всей линии рассуждения.

А.2. Критерии сравнения

Для рассматриваемой задачи предлагается использовать следующие критерии:

1. скорость точечного `longest prefix match`;
2. стоимость вставки одного префикса;
3. стоимость удаления одного префикса;
4. стоимость полной выгрузки всей таблицы;
5. компактность хранения в памяти;

6. удобство конкурентной обработки;
7. сложность реализации и сопровождения в микросервисе общего назначения.

Эти критерии выбраны так, чтобы отражать как алгоритмические, так и системные свойства структуры. Например, высокая скорость `lookup` ничего не говорит о том, насколько легко реализовать безопасный поток подписчиков или дешёвую ресинхронизацию состояния.

А.3. Простые массивы и отсортированные слайсы

Наиболее простая идея – хранить маршруты в линейном массиве или слайсе. В самом примитивном случае поиск требует линейного прохода. В более развитом варианте можно держать массив отсортированным и использовать вспомогательные индексы. Такой подход имеет одно сильное качество: полный `Dump` чрезвычайно дешёв, поскольку данные уже лежат в последовательной форме.

Именно к этой семье по `operational profile` близок `legacy backend`. Его преимущества очевидны:

- очень дешёвый полный снимок;
- простая модель памяти;
- предсказуемое внешнее представление.

Однако фундаментальный недостаток также очевиден: локальное обновление редко бывает по-настоящему локальным. Чтобы сохранить упорядоченность, консистентность и вспомогательные индексы, приходится перестраивать заметную часть состояния. Для `batch`-модели это допустимо, для `streaming`-модели – нет.

A.4. Хеш-таблицы

Чистая хеш-таблица даёт очень привлекательные свойства для точного match по ключу. Вставка и чтение в среднем близки к $O(1)$, код относительно прост, а локальное обновление действительно локально. Но longest prefix match не является точным match по ключу. Для поддержки LPM поверх хеш-таблицы нужно либо хранить множество производных ключей [22], либо перебирать длины масок и проверять кандидатов, либо иметь дополнительную структуру навигации.

Кроме того, чистая hash map плохо подходит для полного обхода в управляемом упорядоченном виде. Все элементы можно перечислить, но их естественный порядок не связан с префиксным пространством. Это особенно неудобно, если Dump должен быть детерминированным и пригодным для downstream-сравнения.

Таким образом, чистая hash map является хорошим вспомогательным элементом, но плохим единственным источником истины для данной задачи.

A.5. Patricia/radix-подходы

Patricia [15] и её современные производные исторически являются очень естественным выбором для prefix-oriented данных. Их главный плюс – структурное соответствие задаче longest prefix match. В дереве легко навигировать по битам или байтам адреса, а компрессия путей делает структуру экономичнее полного trie.

Сильные стороны Patricia/radix:

- естественная поддержка префиксной организации;
- хорошая концептуальная совместимость с LPM;
- понятная теория и богатая историческая база.

Слабые стороны в прикладной постановке могут проявляться в нескольких местах:

- не всегда оптимальное использование памяти в реальных in-memory рантаймах;
- зависимость производительности от конкретной реализации и layout;
- необходимость дополнительной инженерной работы для эффективного Dump и concurrency management.

Если бы задача сводилась только к построению статичного индекса для prefix lookup, Patricia была бы очень сильным претендентом. Но при переходе к многоклиентскому сервису с потоковыми обновлениями возрастают требования к управляемости реализации как системы, а не только как структуры данных.

A.6. LC-trie

LC-trie [17] полезно рассматривать как интеллектуальную эволюцию compressed trie, ориентированную на ускорение поиска. За счёт level compression уменьшается эффективная глубина навигации, а значит и число условных шагов поиска.

Для данной работы LC-trie особенно ценно как контрпример слишком узкой инженерной логики. Если бы целью было только уменьшить latency lookup, именно LC-trie и родственные структуры заслуживали бы приоритетного внимания. Однако здесь встаёт вопрос о цене локальных обновлений, о полном дампе и о простоте интеграции в многопоточный сервис. На этих осях преимущество LC-trie уже не столь очевидно и существенно зависит от конкретной реализации.

A.7. Controlled prefix expansion

Подход controlled prefix expansion [24] демонстрирует, что иногда можно ускорять LPM ценой контролируемого увеличения объёма данных. Это особенно полезно в сценариях, где память относительно дешёва, а latency lookup критична. Для forwarding-plane устройств и специализированных таблиц это может быть очень разумным компромиссом.

В прикладном сервисе соответствия $prefix \rightarrow ASN$ такой подход интересен скорее как теоретический ориентир. Он показывает важный принцип: не всегда стоит минимизировать размер структуры любой ценой; иногда правильнее заплатить памятью за скорость чтения. Однако в потоковой модели этот подход должен быть оценён ещё и по цене обновлений. Если expansion-схема делает локальную вставку дорогой, она снова оказывается не в центре целевого инженерного компромисса.

A.8. Hash-based LPM и binary search on prefix lengths

Работы Waldvogel и других авторов [22, 10] показывают, что longest prefix match можно реализовать не только деревьями, но и комбинацией хеширования, бинарного поиска по длинам префиксов и вспомогательных структур. Для среды, где главным врагом являются лишние memory accesses, эти подходы чрезвычайно убедительны.

Но у них есть тонкое ограничение в рамках данной темы: они прекрасны для статического или слабо изменяемого forwarding-plane представления, однако в сервисе, который одновременно должен:

- принимать поток обновлений;
- отдавать ordered Dump;
- поддерживать подписки;
- интегрироваться в обычную серверную кодовую базу,

такие структуры становятся сложнее в эксплуатации и анализе. Иными словами, выигрыш в чистой lookup-метрике может быть перекрыт ростом сложности состояния как инженерного артефакта.

A.9. Tree Bitmap

Tree Bitmap [7] интересен тем, что пытается совместить достойную скорость lookup, компактность и поддержку инкрементальных обновлений.

Именно это делает его очень важной альтернативой для обсуждения. По духу он ближе к нашей задаче, чем чисто hardware-prefix-expansion схемы.

Тем не менее Tree Bitmap в работе не выбран по двум причинам. Во-первых, его основная литература и инженерная мотивация всё же сильнее привязаны к маршрутизаторным lookup-сценариям, чем к общесервисной mutable state модели. Во-вторых, интеграционная сложность и цена разработки/поддержки в данной кодовой базе для ART-подобного решения оказались ниже.

Это не означает, что Tree Bitmap хуже по существу. Это означает, что он менее естественно вписывается в конкретную комбинацию ограничений текущего проекта.

A.10. Poptrie

Poptrie [3] является особенно сильным современным software baseline. Он показывает, насколько далеко можно продвинуть lookup-скорость на CPU общего назначения, если агрессивно учитывать layout данных и возможности процессорных инструкций (population count). Для работ, где основная цель – быстрый lookup на полном маршрутизаторном столе, Poptrie представляет собой мощный ориентир.

Однако у Poptrie есть тот же важный вопрос, что и у других lookup-first структур: насколько удобно на его основе строить general-purpose mutable service state? Если основная оптимизация достигается плотной компрессией и cache-aware layout, то локальное обновление и concurrent mutation могут становиться более дорогими с инженерной точки зрения.

Поэтому Poptrie полезен как аргумент для раздела «почему новое решение не является абсолютным лидером по чтению», но не как естественный каркас для текущей system architecture.

A.11. ART в сравнительной матрице

Если собрать сравнительную матрицу в качественном виде, получается картина, представленная в табл. 9.

Таблица 9 – Качественное сравнение структур хранения по ключевым критериям

Структура	Lookup	Insert	Delete	Dump	Memory	Concurrency
Линейный массив	низкий	низкий	низкий	высокий	ср.	низкий
Хеш-таблица	высокий*	высокий	высокий	ср.	ср.	ср.
Patricia/radix	высокий	ср.	ср.	ср.	ср.	ср.
LC-trie	высокий	низкий	низкий	ср.	ср.	низкий
Tree Bitmap	высокий	ср.	ср.	ср.	высокий	ср.
Poptrie	высокий	низкий	низкий	ср.	высокий	низкий
ART (шард.)	ср.	высокий	высокий	ср.	высокий	высокий

* Только для exact match; для LPM требуется дополнительная схема (например, binary search on prefix lengths).

Матрица показывает, что ART:

- не лучший во всех lookup-сценариях;
- но очень силён как mutable ordered in-memory index;
- хорошо переносится в сервисную реализацию;
- допускает шардирование и параллелизм;
- удобен как основа для локальных updates и full traversal.

Именно поэтому ART оказывается не «лучшей LPM-структурой вообще», а «наиболее подходящим ядром для данного сочетания требований».

А.12. Вывод по сравнительному анализу

После проведения такого сравнения выбор ART перестаёт выглядеть интуитивным. Он становится следствием формальной логики:

1. целевая система должна поддерживать потоковую актуализацию;
2. потоковая актуализация требует дешёвого локального изменения состояния;
3. дешёвое локальное изменение состояния требует mutable структуры;

4. mutable структура должна при этом поддерживать prefix-oriented lookup и ordered Dump;
5. ART даёт наиболее практичный компромисс под этот набор требований.

Следовательно, выбор ART не является случайным. Он вытекает из постановки задачи и из сравнительного анализа альтернатив.

ПРИЛОЖЕНИЕ Б. Расширенная методика экспериментов и угрозы валидности

Б.1. Операционализация критериев успеха

В любой системной работе по производительности есть риск подменить цель удобными для измерения числами. Поэтому перед обсуждением экспериментов необходимо явно сформулировать критерии успеха и связать их с измерениями.

Для настоящей темы эти критерии таковы:

1. система должна уменьшить стоимость отражения одного изменения маршрута;
2. система должна сохранить *practically acceptable* время чтения;
3. система должна сохранить возможность построения полного снимка;
4. система должна поддерживать потоковый режим как основной операционный сценарий.

Только после такой фиксации становится понятно, почему `Dump`, `Lookup` и `Upsert` являются достаточным минимальным набором метрик: первый отражает сценарий полного снимка для внешнего клиента, второй – сценарий точечного запроса от потребителя, третий – ключевую операцию `streaming`-режима.

Б.2. Репрезентативность синтетических данных

В работе используются синтетические наборы префиксов, поскольку они позволяют воспроизводимо варьировать размер таблицы, число записей и тип адресного семейства. Это удобно и честно для микроизмерений, но создаёт известную угрозу валидности: реальные BGP-таблицы имеют более сложное распределение длин префиксов, плотности кластеризации, уровней агрегации и перекрытий [13, 6].

Поэтому результаты следует интерпретировать как:

- корректные для сравнительного анализа двух реализаций на общем наборе операций;
- достаточно сильные для подтверждения архитектурной гипотезы;
- но не как окончательную оценку поведения на всех возможных реальных интернет-таблицах.

В дальнейшем работу можно усилить измерениями на выгрузках RIS [19] или RouteViews [21] либо на production-дампах GoBGP в анонимизированном виде. Полезным дополнением может оказаться использование RIS-beacons [1] как контрольного потока с известной семантикой.

Б.3. Микробенчмарки и реальная эксплуатация

Микробенчмарки удобны, потому что дают чистое сравнение отдельных операций. Но они не захватывают все факторы production-среды:

- сетевые задержки при общении с GoBGP;
- конкурентные пользовательские запросы и подписки;
- long-lived allocator behaviour и фрагментацию памяти;
- влияние garbage collector на длительном uptime;
- скачки нагрузки на фоне bursty update stream.

Это ограничение не делает эксперименты бесполезными. Оно лишь требует аккуратной интерпретации: измерения показывают не всю систему, а ядро её вычислительной модели.

Б.4. Почему сравнительный эксперимент всё равно валиден

Несмотря на ограничения, сравнительный эксперимент остаётся сильным по двум причинам.

Во-первых, сравниваются две реализации одного и того же abstract contract внутри одной и той же сервисной оболочки. Все различия концентрируются именно в backend-слое, а не в окружающей инфраструктуре.

Во-вторых, именно та операция, которая является целевой для новой архитектуры – `Upsert` – демонстрирует радикальный выигрыш. Положительный результат возникает не в случайной второстепенной метрике, а в главной.

Б.5. Шум и повторяемость измерений

Даже на одном и том же железе microbenchmarks подвержены флуктуациям. На них влияют фоновая активность системы, планировщик потоков, состояние кэшей CPU, поведение памяти, тепловой режим и множество других факторов. Поэтому важно:

- использовать повторные прогоны (`count=2` в работе);
- интерпретировать близкие значения осторожно;
- не делать сильных выводов из различий, лежащих в пределах шума.

Именно поэтому выводы по shard sweep в работе формулируются как качественные, а не как утверждение о том, что «16 шардов всегда лучше 8» или наоборот.

Б.6. Валидность результатов по IPv6

Особого внимания заслуживают результаты IPv6. Здесь разница между backend-вариантами оказывается особенно заметной. Однако и здесь необходимо осторожное чтение.

С одной стороны, ухудшение IPv6 lookup в текущей реализации – реальный факт, который нельзя скрывать. С другой стороны, оно отражает конкретный выбор алгоритма LPM поверх структуры хранения, а не приговор самому подходу «stream + ART». Иначе говоря, измеряется не абстрактный ART вообще, а конкретная реализация longest prefix match над ним.

Б.7. Конструктивные угрозы валидности

Основные угрозы валидности можно сгруппировать так.

Construct validity:

- измеряемые операции упрощают реальную эксплуатационную картину;
- часть сценариев использует синтетические наборы префиксов;
- нагрузка пользователей и нагрузка update stream пока не моделируются одновременно.

Internal validity:

- различия между вариантами могли бы быть искажены деталями реализации, а не только структурой данных;
- некоторые инженерные тюнинги могли быть выполнены несимметрично (например, ART сравнивается без сортировки дампа, но с ней также есть прогон).

External validity:

- результаты не следует напрямую переносить на специализированные forwarding-plane устройства;
- результаты не гарантируют те же относительные величины на другой runtime-платформе или другом объёме таблиц.

Б.8. Почему эти угрозы не разрушают главный вывод

Главный вывод работы устойчив к перечисленным угрозам. Даже если абсолютные цифры изменятся на другом железе или на другом наборе префиксов, качественная разница между:

- полной перестройкой состояния на каждое изменение и
- локальным изменением одного шарда

сохраняется. А именно эта разница и является ядром исследуемого архитектурного перехода.

Б.9. Что стоит добавить в следующем цикле исследований

Для усиления работы и последующих публикаций можно предложить следующие направления развития экспериментальной базы:

1. измерения на реальных снимках RIS [19] и RouteViews [21];
2. долговременные replay-прогоны update streams (часы, дни);
3. смешанная нагрузка `Lookup + Subscribe + updates`;
4. измерения времени восстановления после потери соединения и ресинхронизации;
5. профилирование GC и memory usage на длинных интервалах (pprof);
6. сравнение с дополнительными структурами LPM, например Poptrie-like baseline [3] или вариант, построенный на Tree Bitmap [7];
7. методическое сравнение с подходами в духе controlled prefix expansion [24].

ПРИЛОЖЕНИЕ В. Надёжность, API-семантика и эксплуатационные сценарии

В.1. Почему API является частью исследовательского результата

В инженерных диссертациях API часто воспринимается как сугубо прикладная деталь. Однако в данной работе API тесно связан с самой исследовательской гипотезой. Если гипотеза состоит в том, что состояние должно поддерживаться как `snapshot + stream`, то и интерфейс сервиса обязан отражать эту модель.

Набор операций `GetASNInfo`, `GetPrefixesASTable` и `SubscribePrefixASUpdates` не случаен. Он соответствует трём способам доступа к `materialized state` [11]:

- точечное чтение текущего значения;
- чтение полного среза;
- подписка на инкрементальные изменения.

Именно эта тройка делает внешний контракт сервиса согласованным с его внутренней моделью.

В.2. Семантика lookup-запроса

`GetASNInfo` решает наиболее частую и наиболее простую с точки зрения клиента задачу: по конкретному IP-адресу определить наиболее специфичный префикс и связанный с ним ASN. Для downstream-клиента эта операция должна быть:

- быстрой;
- предсказуемой;

- не требующей знания внутренней структуры таблицы.

Внутренне сервис может реализовывать `longest prefix match` сложным образом, но наружу он обязан предоставлять простую и стабильную семантику: «вернуть лучшее известное на текущий момент соответствие».

В.3. Семантика полного снимка

`GetPrefixesASTable` на первый взгляд выглядит самой простой операцией: просто вернуть всю таблицу. На деле именно здесь скрываются важные вопросы семантики.

Во-первых, «вся таблица» должна быть согласованным `snapshot`, а не набором записей, собранных в середине конкурентных изменений.

Во-вторых, клиенту обычно нужна таблица в форме, пригодной для дальнейшего анализа, а не внутреннее представление дерева.

В-третьих, для крупных таблиц важно понимать стоимость операции и не считать её «дешёвой по умолчанию» – особенно в режиме шардированного ART, где `Dump` обходит несколько ART-деревьев и выполняет аллокации.

Это делает `GetPrefixesASTable` не просто сервисным `convenience method`, а отдельным операционным сценарием со своей ценой и своей архитектурной значимостью.

В.4. Семантика подписки на обновления

`SubscribePrefixASUpdates` наиболее полно выражает `streaming`-природу системы. С точки зрения клиента этот метод означает: «после установления подписки я хочу получать все изменения состояния в порядке их применения локальным сервисом».

Здесь возникают важные эксплуатационные вопросы:

- нужно ли гарантировать `delivery exactly once` или `at least once` [12];
- что считается границей начала подписки;
- как клиент восстанавливается после разрыва;

- нужно ли клиенту предварительно получить полный снимок.

В текущей архитектуре естественная модель выглядит так:

1. при необходимости клиент сначала получает полный snapshot;
2. затем открывает stream;
3. stream используется как инкрементальная дельта поверх уже известного снимка.

Эта модель хорошо согласуется с общей идеей snapshot + stream и легко объясняется downstream-командам, в особенности тем, кто знаком с шаблонами log-oriented архитектур [11].

В.5. Поведение при отказах

Надёжность stream-oriented сервиса определяется не только тем, как он работает в штатном режиме, но и тем, как он переживает сбои. Для данной системы критичны как минимум три класса отказов:

1. временная потеря связи с GoBGP;
2. внутренняя ошибка обработки update-события;
3. разрыв клиентской подписки.

Стратегии работы с этими отказами различны.

При потере связи с GoBGP сервис обязан считать поток потенциально неполным и инициировать reconnect-and-resync (см. рис. 4).

При внутренней ошибке обработки события сервис должен избегать продолжения работы в сомнительном состоянии. В production-ориентированной реализации предпочтительнее пересобрать состояние заново, чем продолжать эксплуатацию с возможной скрытой рассогласованностью.

При разрыве клиентской подписки ответственность переносится на клиента: он либо повторно подписывается и принимает модель «сначала снимок, потом дельта», либо строит собственные механизмы компенсации.

В.6. Эксплуатационный баланс между snapshot и stream

Одна из важных эксплуатационных идей работы состоит в том, что snapshot и stream не конкурируют друг с другом. Они не являются двумя разными режимами, из которых нужно выбрать один. Напротив, они образуют целостную модель:

Snapshot нужен:

- при старте;
- при ресинхронизации;
- для некоторых внешних клиентов;
- для валидации и отладки.

Stream нужен:

- для поддержания свежести состояния;
- для локальных изменений;
- для downstream-reactive processing;
- для минимизации общей стоимости обновления.

Поэтому архитектурно правильно не противопоставлять snapshot и stream, а рассматривать их как две фазы жизни одного и того же состояния [5, 2].

В.7. Безопасность и контроль доступа

Хотя сама работа не фокусируется на вопросах аутентификации и авторизации, при промышленном внедрении сервис неизбежно становится частью доверенного внутреннего контура. Это означает, что для production-эксплуатации полезны:

- разграничение доступа к полному Dump и к точечному Lookup;

- отдельный контроль на операции подписки;
- аудит подключений;
- rate limiting для особенно дорогих операций.

Эти соображения не влияют на основную исследовательскую гипотезу, но определяют готовность системы к реальному использованию.

В.8. Наблюдаемость и профилирование

Важный практический аспект разработанного сервиса – наличие средств диагностики. Возможность включить `pprof` и исследовать поведение отдельных компонентов особенно полезна для систем, где компромиссы между `Lookup`, `Dump` и `Upsert` видны только на профилях памяти и CPU. С точки зрения системной инженерии это означает, что сервис строится не как «чёрный ящик», а как наблюдаемая подсистема, пригодная для дальнейшей эволюции. Концептуально это близко к идее BMP [23]: маршрутизационное состояние и сервисы, его обрабатывающие, должны быть наблюдаемы как отдельный поток.

В.9. Потенциальное расширение API

В будущем API сервиса может быть расширен в нескольких направлениях:

- выдача метаданных о свежести локального состояния (`timestamp` последнего обновления, `lag` относительно источника);
- получение дифференциального `snapshot` за интервал времени;
- выдача статистики по `flapping prefixes` (с использованием идей RFC 2439 [25]);
- запросы по ASN с получением всех известных префиксов (обратное отношение);

- диагностические методы для состояния sync/watch-loop;
- поддержка фильтрации по <AFI, SAFI> [16] в потоке подписки.

Все эти направления логически следуют из уже реализованного ядра и усиливают ценность сервиса как общей платформы маршрутизационной аналитики.

В.10. Итог по эксплуатационной части

Эксплуатационная ценность разработанного решения состоит не только в том, что оно «быстро обновляет дерево». Она состоит в том, что построен сервис с корректной state model, ясным API и внятным поведением при отказах. Именно сочетание алгоритмического ядра и системной оболочки делает результат диссертационным, а не просто локальной оптимизацией структуры данных.

ПРИЛОЖЕНИЕ Г. Практическая интеграция и инфраструктурный контекст

Г.1. Место сервиса в общей продуктовой картине

Разработанный микросервис проектировался не как абстрактный исследовательский прототип, а как компонент продуктовой инфраструктуры компании, в которой одновременно эксплуатируются системы защиты от DDoS-атак (BIFIT MITIGATOR) и системы анализа сетевого трафика (BIFIT COLLECTOR). Обе категории продуктов полагаются на актуальное знание принадлежности IP-адресов автономным системам, но используют его по-разному:

- для систем защиты ASN-метка является сигналом, на основе которого принимаются решения о фильтрации, скоринге и распределении нагрузки;
- для систем анализа ASN-метка участвует в построении сводных отчётов, в детектировании аномалий и в обогащении flow-записей.

Принципиально важно, что обоим продуктам нужен один и тот же ответ на вопрос «к какой ASN принадлежит данный IP», но раньше каждый из них извлекал этот ответ собственным способом, с собственным жизненным циклом обновления и собственной моделью консистентности.

Г.2. Эффект унификации источника данных

Появление общего сервиса даёт несколько системных эффектов, не сводимых только к ускорению одной операции.

Во-первых, исчезает источник скрытого расхождения данных. Когда два продукта строят свои таблицы независимо, их представления о принадлежности одного и того же IP могут разойтись, что особенно неприятно при разборе инцидентов, где требуется единая трактовка наблюдений. Унификация источника данных снимает этот класс проблем по построению.

Во-вторых, упрощается операционная модель. Поддерживать актуальной одну подсистему дешевле, чем поддерживать актуальными несколько независимых реализаций одной и той же функции. Это особенно заметно при изменениях в источнике данных или в формате представления BGP.

В-третьих, появляется естественная точка для расширения. Например, можно добавить в сервис способность возвращать дополнительные маршрутизационные атрибуты, не меняя при этом архитектуру каждого продукта-потребителя. С точки зрения *evolvability* это очень существенный сдвиг по сравнению с разрозненной реализацией.

Г.3. Снижение совокупной стоимости владения

Сравнительная экономия инфраструктурного контура от появления общего сервиса заметна сразу по нескольким направлениям:

- на уровне CPU – благодаря уменьшению числа повторных пересборок таблицы в каждом продукте;
- на уровне памяти – благодаря отсутствию дублирующихся *in-memory* представлений одного и того же состояния;
- на уровне сети – благодаря тому, что подключения к источнику маршрутных данных консолидируются в одной точке;
- на уровне эксплуатации – благодаря единым средствам наблюдаемости, профилирования и диагностики;
- на уровне разработки – благодаря единой кодовой базе для поддержки маршрутизационного отображения.

Каждый из этих пунктов в отдельности можно рассматривать как чисто инженерное улучшение, но в совокупности они формируют качественно другую модель эксплуатации.

Г.4. Влияние на жизненный цикл продуктов

Помимо непосредственной экономии, сервис влияет на скорость эволюции продуктов. Раньше любое изменение в способе получения ASN-данных приводило к одинаковым изменениям в нескольких кодовых базах. Теперь оно локализовано в одном сервисе, а продукты-потребители получают изменение «бесплатно» в рамках обычного обновления зависимостей API.

Это особенно важно для следующих типичных сценариев:

- смена источника BGP (например, переход на другой узел GoBGP или подключение дополнительного источника);
- изменение модели обработки IPv6 (например, более тонкая сегментация /64);
- введение нового downstream-клиента (например, новой аналитической подсистемы);
- появление дополнительных метаданных в маршрутных записях (например, длины AS-path или признаков агрегации, см. [8]).

Г.5. Связь с теоретической моделью

Эффект унификации источника, описанный выше, является частным проявлением более общего архитектурного принципа: один производитель истины, несколько потребителей, общий поток обновлений. Этот принцип явно проводится в работах по log-oriented архитектурам [12, 11], в моделях stateful streaming [5, 2] и в наблюдаемости BGP-плана через BMP [23].

Таким образом, разработанный сервис не только решает прикладную задачу, но и встраивается в более широкую инженерную традицию проектирования систем работы с потоками изменений состояния.

Г.6. Возможные пути будущей интеграции

Текущая интеграция сервиса с продуктами компании является базовой: клиенты обращаются к нему по gRPC и используют возвращаемые ответы

напрямую. В будущем разумны следующие направления развития интеграционного контура:

1. Кэширование на стороне клиента с инвалидацией через подписку `SubscribePrefixASUpdates`.
2. Промежуточные адаптеры, преобразующие поток обновлений в другие форматы (например, в очередь сообщений или в Kafka-топик [12]).
3. Шардированное обслуживание, в котором несколько экземпляров сервиса делят между собой нагрузку по AFI/SAFI или по диапазонам ASN.
4. Многослойная топология, в которой продуктовые сервисы подписываются на агрегированный поток событий, а не каждый раз вычисляют lookup самостоятельно.
5. Интеграция с внешними route collectors [19, 21] для верификации и обогащения локального состояния.

Г.7. Итог приложения

С точки зрения практической инженерии разработанный сервис выполняет три функции одновременно:

- служит общей платформой для нескольких продуктов;
- обеспечивает свежесть состояния на уровне реального времени;
- даёт инфраструктурный фундамент для последующих расширений в области сетевой аналитики и наблюдаемости.

В этом смысле работа имеет не только исследовательское, но и непосредственное эксплуатационное значение, что подтверждает заявленную во введении практическую значимость и сводит воедино архитектурный и алгоритмический результаты диссертации.